List of backend tests

All 1026 automated backend tests of RWCrew, with a short description · branch `develop`, 9 November 2026 · companion to *RWCrew-Testaanpak.pdf*

How to read this list. The tests are grouped per area of the app, then per test file (with that file's own introduction). Every line is one test. **The bold sentence is the test's name written out** – tests here are deliberately named as a sentence that states the rule they check. A grey line under it is the explanation written in the test itself, where there is one. Some tests run several times with different values (e.g. once per module or per language); they are listed once, with the number of runs on the right. Those runs add up to 1026. The small grey code text is the test's real name in the code (to search for it or run it alone with `pytest -k <name>`).

Words used a lot

200/201/204 = request succeeded (OK / created / deleted)
400 = refused (a rule says no)
401 = not logged in **403** = logged in but no right
404 = not found (or hidden from you)
422 = invalid input

Screen / permission = a right per screen: view, create, edit, delete
Super admin = decides who may open which module
Bypass = super admin passes every screen check
Sync screens = the module's screen list is written to the database
Snapshot = a value copied at that moment, so it stays as it was

Area	Files	Tests
General	1	2
Core helpers	2	5
Login & access rights	7	72
Help manuals	1	28
Mass test tool	1	4
Scans / TagScan (module-1)	13	203
Wristband scans (shared)	1	14
KarTracker (module-2)	21	184
Interventies (module-3)	5	43
StockMaster (module-4)	12	123
Atelier (module-5)	12	93
Walkies (module-6)	2	17
Altsien Select (module-8)	10	90
MasterData (module-9)	13	145
Mobile App (module-10)	1	3
Total	**102**	**1026**

General

2 tests

The app answers its health check.

`test_health.py` 2

Tests for /api/health: the basic "is the API up" check, which also reports the deployed version (commit date · short hash) set by deploy.sh.

1 **Health reports dev without a deployed version.** 1
`test_health_reports_dev_without_a_deployed_version`

2 **Health reports the deployed version.** 1
`test_health_reports_the_deployed_version`

Core helpers

5 tests

Mail sending and Belgian time, used by every module.

`test_mail.py` 2

app/core/mail.py: the CC list travels as Resend's "cc", only on the first chunk of a long "to" list (so a CC address never gets the mail twice). httpx.post is replaced by a collector, so nothing leaves the machine.

3	**CC only on the first chunk.** `test_cc_only_on_the_first_chunk`	1
4	**No CC key without CC.** `test_no_cc_key_without_cc`	1

`test_timezone.py` 3

Tests for the Belgian-time helpers every PDF, mail and dashboard uses (app/core/timezone.py): summer and winter time, and naive values (how SQLite in tests returns stored UTC moments).

5	**Summer time is UTC plus two and can change the date.** `test_summer_time_is_utc_plus_two_and_can_change_the_date`	1
6	**Winter time is UTC plus one.** `test_winter_time_is_utc_plus_one`	1
7	**A naive value is treated as UTC.** `test_a_naive_value_is_treated_as_utc`	1

Login & access rights

72 tests

Login, sessions, passwords, module access granted by the super admin, login history, changelog.

`test_access.py` 20

These tests check the access-rights system end-to-end: - Only the super admin can grant/revoke which modules a user can open. - A user without access to a module is refused when they try to use it. - A module's own admin (not the super admin) can manage roles within that one module, but nowhere else.

8	**Non super admin cannot list users.** `test_non_super_admin_cannot_list_users`	1
9	**Super admin can grant module access.** `test_super_admin_can_grant_module_access`	1
10	**Re granting the same access a user already has is idempotent.** Regression test: sending the exact same, unchanged module_keys list twice in a row used to fail with a database error, because the endpoint deletes and re-adds every grant even when nothing actually changed — see the "db.flush()" fix in set_user_module_access. `test_re_granting_the_same_access_a_user_already_has_is_idempotent`	1
11	**Super admin can revoke previously granted access.** `test_super_admin_can_revoke_previously_granted_access`	1
12	**Super admin does not automatically have module access.** Being super admin only grants the power to manage access for everyone — it does not, by itself, open every module's tile. Uses module-7 (the last placeholder) since every other module now has its own bespoke router/permission system, tested separately. `test_super_admin_does_not_automatically_have_module_access`	1
13	**User without access cannot open a module.** `test_user_without_access_cannot_open_a_module`	1
14	**User with granted access can open the module.** `test_user_with_granted_access_can_open_the_module`	1
15	**Module admin can assign roles in their own module.** `test_module_admin_can_assign_roles_in_their_own_module`	1
16	**Module reader cannot assign roles.** `test_module_reader_cannot_assign_roles`	1
17	**Being admin of one module does not grant admin of another.** `test_being_admin_of_one_module_does_not_grant_admin_of_another`	1

18	**Super admin can manage roles in any module.** `test_super_admin_can_manage_roles_in_any_module`	1
19	**Super admin can delete a user.** `test_super_admin_can_delete_a_user`	1
20	**Deleting a user also removes their module access.** `test_deleting_a_user_also_removes_their_module_access`	1
21	**Super admin cannot delete their own account.** `test_super_admin_cannot_delete_their_own_account`	1
22	**Non super admin cannot delete users.** `test_non_super_admin_cannot_delete_users`	1
23	**Creating a user can set the kernlid flag and phone.** `test_creating_a_user_can_set_the_kernlid_flag_and_phone`	1
24	**Super admin can update flags and phone.** `test_super_admin_can_update_flags_and_phone`	1
25	**Super admin cannot remove their own super admin flag.** `test_super_admin_cannot_remove_their_own_super_admin_flag`	1
26	**Non super admin cannot update users.** `test_non_super_admin_cannot_update_users`	1
27	**Kernlid lookup lists only active flagged users.** `test_kernlid_lookup_lists_only_active_flagged_users`	1

`test_auth.py` 10

These tests check the login-related API endpoints end-to-end: logging in with correct/incorrect credentials, reading "who am I", refreshing a session, and logging out. They use the FastAPI TestClient, which sends real HTTP requests to our app (backed by a temporary test database, see conftest.py) and follows cookies the same way a browser would.

28	**Login with correct credentials succeeds.** `test_login_with_correct_credentials_succeeds`	1
29	**Login with wrong password is rejected.** `test_login_with_wrong_password_is_rejected`	1
30	**Login with unknown email is rejected.** `test_login_with_unknown_email_is_rejected`	1
31	**Login for a disabled account is rejected.** `test_login_for_a_disabled_account_is_rejected`	1
32	**Me without logging in is rejected.** `test_me_without_logging_in_is_rejected`	1
33	**Me after logging in returns the logged in user.** `test_me_after_logging_in_returns_the_logged_in_user`	1
34	**Logout then me is rejected again.** `test_logout_then_me_is_rejected_again`	1
35	**Refresh issues a new working session.** `test_refresh_issues_a_new_working_session`	1
36	**Reusing an already rotated refresh token is rejected.** This is the "reuse detection" behaviour: once a refresh token has been used once, using that SAME token text again must fail — it was already replaced by a new one. `test_reusing_an_already_rotated_refresh_token_is_rejected`	1
37	**Refresh never extends the login session.** The session ends a fixed number of hours after the password was entered; renewing must keep that end, and once it has passed the user has to log in again. `test_refresh_never_extends_the_login_session`	1

`test_changelog.py` 15

These tests check the changelog behind the super admin's "Wijzigingen" screen: only the super admin may read it, the lines come newest first, the file is validated strictly, and a GitHub outage doesn't lose the list. GitHub itself is never called: httpx.get is replaced in every test.

38	**Non super admin cannot read the changelog.** `test_non_super_admin_cannot_read_the_changelog`	1

39	**Super admin gets the newest line on top.** `test_super_admin_gets_the_newest_line_on_top`	1
40	**Unreachable github without a copy gives 503 (unavailable).** `test_unreachable_github_without_a_copy_gives_503`	1
41	**List is cached for a short while.** `test_list_is_cached_for_a_short_while`	1
42	**Last good copy is served when github fails.** `test_last_good_copy_is_served_when_github_fails`	1
43	**Invalid file keeps the last good copy.** `test_invalid_file_keeps_the_last_good_copy`	1
44	**Invalid file without a copy is unavailable.** `test_invalid_file_without_a_copy_is_unavailable`	1
45	**Parse refuses invalid lines.** Runs for: line0, line1, line2, line3, line4, line5, line6 `test_parse_refuses_invalid_lines`	×7
46	**Parse refuses duplicate IDs and broken JSON.** `test_parse_refuses_duplicate_ids_and_broken_json`	1

`test_login_history.py` 13

These tests check the login history feature: every login attempt (successful or not) is recorded in Landing_login_history, and the super-admin-only "Login History" endpoint lists, paginates, and date-filters those rows correctly.

47	**Successful login is recorded.** `test_successful_login_is_recorded`	1
48	**Wrong password is recorded with the matched user.** `test_wrong_password_is_recorded_with_the_matched_user`	1
49	**Unknown email is recorded with no user.** `test_unknown_email_is_recorded_with_no_user`	1
50	**Website login from a pc is recorded as web and desktop.** `test_website_login_from_a_pc_is_recorded_as_web_and_desktop`	1
51	**Phone section login is recorded as pwa and mobile.** `test_phone_section_login_is_recorded_as_pwa_and_mobile`	1
52	**Failed attempt also records source and device.** `test_failed_attempt_also_records_source_and_device`	1
53	**An unknown client value is rejected.** `test_an_unknown_client_value_is_rejected`	1
54	**A very long user agent is cut to the column size.** `test_a_very_long_user_agent_is_cut_to_the_column_size`	1
55	**Classify device.** `test_classify_device`	1
56	**Non super admin cannot list login history.** `test_non_super_admin_cannot_list_login_history`	1
57	**Super admin sees login history newest first.** `test_super_admin_sees_login_history_newest_first`	1
58	**Login history supports pagination.** `test_login_history_supports_pagination`	1
59	**Login history supports date range filtering.** `test_login_history_supports_date_range_filtering`	1

`test_password.py` 4

These tests check the two ways a password can change: a super admin resetting someone else's (PATCH /api/admin/users/{id}) and a user changing their own (POST /api/auth/change-password). Both must end old sessions.

60	**Admin can reset a password and old sessions end.** `test_admin_can_reset_a_password_and_old_sessions_end`	1
61	**Reset password needs super admin and min length.** `test_reset_password_needs_super_admin_and_min_length`	1

62 **User can change own password.** 1
`test_user_can_change_own_password`

63 **Change password rejects wrong current or same password.** 1
`test_change_password_rejects_wrong_current_or_same_password`

`test_security.py` 6

These tests check the lowest-level security building blocks in app/core/security.py: password hashing/verification, and creating and reading login tokens (JWTs). Everything else (login, access rights) depends on these working correctly, so we test them in isolation first.

64 **Hashing the same password twice gives different hashes.** 1
Two random salts should make the stored hashes look different, even though both were made from the exact same password.
`test_hashing_the_same_password_twice_gives_different_hashes`

65 **Verify password accepts the correct password.** 1
`test_verify_password_accepts_the_correct_password`

66 **Verify password rejects the wrong password.** 1
`test_verify_password_rejects_the_wrong_password`

67 **Access token round trip returns the same user ID.** 1
`test_access_token_round_trip_returns_the_same_user_id`

68 **A refresh token is rejected where an access token is expected.** 1
This guards against a security bug where a long-lived refresh token could be used directly as if it were a short-lived access token.
`test_a_refresh_token_is_rejected_where_an_access_token_is_expected`

69 **A tampered token is rejected.** 1
`test_a_tampered_token_is_rejected`

`test_token_auth.py` 4

The Raspberry Pi app's login: the same accounts and tokens as the website, but handed over in the response body and sent back as an "Authorization: Bearer" header (it has no cookies). Checks the login, the Bearer way in, the rotated refresh, logout and the login history's "pi".

70 **Token login gives tokens without cookies.** 1
`test_token_login_gives_tokens_without_cookies`

71 **Wrong password is rejected.** 1
`test_wrong_password_is_rejected`

72 **Bearer header logs the request in.** 1
`test_bearer_header_logs_the_request_in`

73 **Refresh rotates and logout ends the session.** 1
`test_refresh_rotates_and_logout_ends_the_session`

Help manuals 28 tests

The PDF user manuals of every module.

`test_help.py` 28

The module help manuals: the Markdown parser, the content of every manual (chapters point to real screens, both languages match, screenshots exist) and the PDF builder.

74 **Parser reads every block kind.** 1
`test_parser_reads_every_block_kind`

75 **Manual content is consistent.** ×8
Runs for: module-1, module-2, module-3, module-4, module-5, module-6, module-8, module-9
`test_manual_content_is_consistent`

76 **Every content folder is tested.** 1
`test_every_content_folder_is_tested`

77 **Chapters follow the viewable screens.** 1
`test_chapters_follow_the_viewable_screens`

78 **PDF renders full and per topic.** ×16
Runs for: module-1-nl, module-1-en, module-2-nl, module-2-en, module-3-nl, module-3-en, module-4-nl, module-4-en, module-5-nl, module-5-en, module-6-nl, module-6-en …
`test_pdf_renders_full_and_per_topic`

79 **Hidden or unknown topic is refused.** 1
`test_hidden_or_unknown_topic_is_refused`

Mass test tool 4 tests

Loading last year's lists through the API.

`test_masstest.py` 4

The mass test (backend/masstest): converting last year's Uitgifte/Inname list into the load workbook, the solution rules and texts, the report, and a full run against the real app (in-memory database) through an httpx transport that forwards to the TestClient — what loads, what is skipped and logged with which code, that a second run loads nothing twice, that a dry run writes nothing, and that back-dated bookings …

80 **Convert groups transactions and logs unreadable rows.** 1
`test_convert_groups_transactions_and_logs_unreadable_rows`

81 **Rules texts and report.** 1
`test_rules_texts_and_report`

82 **Full run loads logs and never duplicates.** 1
`test_full_run_loads_logs_and_never_duplicates`

83 **Run stops before writing without the rights.** 1
`test_run_stops_before_writing_without_the_rights`

Scans / TagScan (module-1) 203 tests

RFID CSV intake, tags, scanners, processing of scanned lines.

`test_auto_scan.py` 7

Tests for TagScan's automatic background Scan (auto_scan.py): its settings on the Settings screen (defaults, validation, permissions) and one run of the job — scans when on and due, does nothing when switched off or when the interval hasn't passed yet.

84 **Settings show auto scan defaults when nothing saved.** 1
`test_settings_show_auto_scan_defaults_when_nothing_saved`

85 **Saving auto scan settings.** 1
`test_saving_auto_scan_settings`

86 **Auto scan interval must be at least ten seconds.** 1
`test_auto_scan_interval_must_be_at_least_ten_seconds`

87 **View only permission cannot change auto scan.** 1
`test_view_only_permission_cannot_change_auto_scan`

88 **Run scans new CSV and records the run.** 1
`test_run_scans_new_csv_and_records_the_run`

89 **Run does nothing when switched off.** 1
`test_run_does_nothing_when_switched_off`

90 **Run waits for the interval.** 1
`test_run_waits_for_the_interval`

`test_dashboard.py` 7

These tests check TagScan's landing dashboard: aggregate KPI stats gated by plain module access (no specific screen permission, matching the unconditional placeholder page it replaces).

91 **Dashboard requires module access.** 1
`test_dashboard_requires_module_access`

92 **Dashboard with no tags returns all zero stats.** 1
`test_dashboard_with_no_tags_returns_all_zero_stats`

93 **Dashboard unreaded tags count reflects CSV files in unreaded tags.** 1
`test_dashboard_unreaded_tags_count_reflects_csv_files_in_unreaded_tags`

94	**Dashboard computes counts and status breakdown.** `test_dashboard_computes_counts_and_status_breakdown`	1
95	**Dashboard registered this month only counts current month.** `test_dashboard_registered_this_month_only_counts_current_month`	1
96	**Dashboard top products ordered by tag count.** `test_dashboard_top_products_ordered_by_tag_count`	1
97	**Dashboard registrations by week buckets correctly.** `test_dashboard_registrations_by_week_buckets_correctly`	1

`test_device_intake.py` 24

Tests for the device-facing CSV push endpoint (POST /api/public/tagscan-intake) — API-key auth (not user login), landing a file into "Unreaded Tags", and the three-way duplicate check that makes a retried upload safe (see app/modules/module_1/device_router.py).

98	**Upload without API key header is rejected.** `test_upload_without_api_key_header_is_rejected`	1
99	**Upload with malformed API key returns 401 (not logged in).** `test_upload_with_malformed_api_key_returns_401`	1
100	**Upload with wrong secret returns 401 (not logged in).** `test_upload_with_wrong_secret_returns_401`	1
101	**Upload for unknown scanner ID returns 401 (not logged in).** `test_upload_for_unknown_scanner_id_returns_401`	1
102	**Upload with non CSV filename returns 400 (refused).** `test_upload_with_non_csv_filename_returns_400`	1
103	**Oversized upload returns 413.** `test_oversized_upload_returns_413`	1
104	**Successful upload lands the file and updates last used.** `test_successful_upload_lands_the_file_and_updates_last_used`	1
105	**Reuploading an unreaded file is a safe duplicate.** `test_reuploading_an_unreaded_file_is_a_safe_duplicate`	1
106	**Reuploading a file already moved to read tags is a safe duplicate.** `test_reuploading_a_file_already_moved_to_read_tags_is_a_safe_duplicate`	1
107	**Same filename with different content is stored not dropped.** `test_same_filename_with_different_content_is_stored_not_dropped`	1
108	**Retrying a content conflicting upload is a safe duplicate.** `test_retrying_a_content_conflicting_upload_is_a_safe_duplicate`	1
109	**Reuploading a file already logged as header data is a safe duplicate.** `test_reuploading_a_file_already_logged_as_header_data_is_a_safe_duplicate`	1
110	**Upload rejects a filename trying to escape the intake folder.** `test_upload_rejects_a_filename_trying_to_escape_the_intake_folder`	1
111	**CSV for the other environment is forwarded not stored.** `test_csv_for_the_other_environment_is_forwarded_not_stored`	1
112	**CSV for this environment is stored here.** `test_csv_for_this_environment_is_stored_here`	1
113	**CSV without a known mode is stored here.** Runs for: Scanner,EPC\nScan-01,ABC\n, Scanner,Mode,EPC\nScan-01,,ABC\n, Scanner,Mode,EPC\nScan-01,staging,ABC\n `test_csv_without_a_known_mode_is_stored_here`	×3
114	**Routing is off when this environment has no name.** `test_routing_is_off_when_this_environment_has_no_name`	1
115	**An already forwarded file is always stored.** `test_an_already_forwarded_file_is_always_stored`	1
116	**Forwarding without configuration returns 503 (unavailable).** `test_forwarding_without_configuration_returns_503`	1
117	**Forwarding refused by the other side returns 502.** `test_forwarding_refused_by_the_other_side_returns_502`	1

118	**Forwarding network error returns 502.** `test_forwarding_network_error_returns_502`	1
119	**Read CSV mode and normalize environment.** `test_read_csv_mode_and_normalize_environment`	1

`test_file_browser.py` 9

Tests for TagScan's read-only Explorer-style file browser (the Dashboard screen's real content): folder tree, per-folder file listing, file content preview and deleting files — gated by the "tagscan.dashboard" view (delete: delete) permission, plus the path-traversal guard in file_browser.py.

120	**User without dashboard permission gets 403 (forbidden) on every endpoint.** `test_user_without_dashboard_permission_gets_403_on_every_endpoint`	1
121	**Folder tree reflects the real directory structure.** `test_folder_tree_reflects_the_real_directory_structure`	1
122	**Listing files returns only direct children.** `test_listing_files_returns_only_direct_children`	1
123	**File content preview returns the real text.** `test_file_content_preview_returns_the_real_text`	1
124	**Path traversal attempt is rejected.** `test_path_traversal_attempt_is_rejected`	1
125	**Missing file returns 404 (not found).** `test_missing_file_returns_404`	1
126	**Deleting files removes only the given files.** `test_deleting_files_removes_only_the_given_files`	1
127	**Deleting with one bad path deletes nothing.** `test_deleting_with_one_bad_path_deletes_nothing`	1
128	**View only dashboard permission cannot delete files.** `test_view_only_dashboard_permission_cannot_delete_files`	1

`test_header_data.py` 27

Tests for the "Tag Headerdata" screen: scanning TagScan's "Unreaded Tags" intake subfolder for CSV files, logging each one exactly once (id, filename, created_at, line_count), and moving it into "Read Tags" — gated by the "tagscan.tag-headerdata" permission, independently of every other module_1 screen.

129	**User without permission gets 403 (forbidden) on list and scan.** `test_user_without_permission_gets_403_on_list_and_scan`	1
130	**Scan with no unreaded tags folder returns empty result.** No scan_dirs fixture here — the source root exists, but "Unreaded Tags" itself does not. `test_scan_with_no_unreaded_tags_folder_returns_empty_result`	1
131	**Scan logs new CSV files counts data lines and moves them.** `test_scan_logs_new_csv_files_counts_data_lines_and_moves_them`	1
132	**Line count excludes the header row and a trailing blank line.** 1 header row + 2 data rows + 1 trailing blank line = 4 physical lines, but only 2 rows are ever logged to Tag Linedata — line_count must match that 2, not the raw physical line count. `test_line_count_excludes_the_header_row_and_a_trailing_blank_line`	1
133	**Scan resolves header scanner snapshot from the first CSV row.** `test_scan_resolves_header_scanner_snapshot_from_the_first_csv_row`	1
134	**Scan leaves header scanner snapshot null when unmatched.** `test_scan_leaves_header_scanner_snapshot_null_when_unmatched`	1
135	**Synchro re resolves a renamed scanners header snapshot.** The "Synchro" action lives on the Tag Linedata screen, gated by "tagscan.tag-linedata" — a super admin is used here rather than _viewer_client (which only grants "tagscan.tag-headerdata") so the test can exercise both the scan and the sync in one flow. `test_synchro_re_resolves_a_renamed_scanners_header_snapshot`	1
136	**Scan creates read tags folder if missing.** `test_scan_creates_read_tags_folder_if_missing`	1
137	**Scan ignores non CSV files.** `test_scan_ignores_non_csv_files`	1

138	**Second scan does not relog an already processed file.** `test_second_scan_does_not_relog_an_already_processed_file`	1
139	**Duplicate filename race is handled via integrity error.** A genuine TOCTOU race: another request's INSERT lands between this scan's "already logged?" pre-check and its own commit. `test_duplicate_filename_race_is_handled_via_integrity_error`	1
140	**View only permission cannot trigger scan.** `test_view_only_permission_cannot_trigger_scan`	1
141	**Super admin can scan without explicit role.** `test_super_admin_can_scan_without_explicit_role`	1
142	**List header data ordered newest first.** `test_list_header_data_ordered_newest_first`	1
143	**Delete removes header and its line rows and allows relogging.** `test_delete_removes_header_and_its_line_rows_and_allows_relogging`	1
144	**Deleting an unknown header row returns 404 (not found).** `test_deleting_an_unknown_header_row_returns_404`	1
145	**View only permission cannot delete.** `test_view_only_permission_cannot_delete`	1
146	**PDF download returns a valid PDF.** `test_pdf_download_returns_a_valid_pdf`	1
147	**PDF content disposition escapes special characters in filename.** A raw quote/backslash can never come from a real Windows-sourced filename, but the header-building code must never produce a broken/injectable Content-Disposition header for any string it's given — insert one directly rather than via a real scan. `test_pdf_content_disposition_escapes_special_characters_in_filename`	1
148	**PDF download for unknown header returns 404 (not found).** `test_pdf_download_for_unknown_header_returns_404`	1
149	**User without permission cannot download PDF.** `test_user_without_permission_cannot_download_pdf`	1
150	**List header data breaks a timestamp tie by newest ID.** `test_list_header_data_breaks_a_timestamp_tie_by_newest_id`	1
151	**Scan sets registered on to the moment of saving.** `test_scan_sets_registered_on_to_the_moment_of_saving`	1
152	**Synchro fills an empty header scanner from its lines.** A header logged before the header-level scanner columns existed: its own scanner is empty, but its lines carry the device name. `test_synchro_fills_an_empty_header_scanner_from_its_lines`	1
153	**Scan snapshots the tags kar number on lines and header.** `test_scan_snapshots_the_tags_kar_number_on_lines_and_header`	1
154	**Scan snapshots the tags product ID for the product link.** `test_scan_snapshots_the_tags_product_id_for_the_product_link`	1
155	**Synchro picks up a kar linked after the scan.** `test_synchro_picks_up_a_kar_linked_after_the_scan`	1

`test_help.py` 2

TagScan's help manual endpoint (GET /help/pdf): module access is needed, and a topic of a screen the user can't view is refused — the general chapters (how it works, the overview) are open to everyone with access.

156	**Help needs module access.** `test_help_needs_module_access`	1
157	**Topics follow the role.** `test_topics_follow_the_role`	1

`test_line_data.py` 27

Tests for the "Tag Linedata" screen: every CSV data line produced by a Tag Headerdata scan becomes its own row, linked to the header row it came from and enriched with a snapshot of its matching TagManagement tag (if any) — gated by "tagscan.tag-

linedata", independently of every other module_1 screen. Also covers the manual "cancel" action, the only way a line's status ever becomes "cancelled".

158	**Scan handles a leading utf8 bom without breaking column matching.** Excel/scanner software commonly writes a UTF-8 BOM at the start of a CSV. Decoded as plain utf-8, that BOM glues onto the first header name ("Scanner"), which would then never match COLUMN_SCANNER and silently leave every row's "scanner" field None. `test_scan_handles_a_leading_utf8_bom_without_breaking_column_matching`	1
159	**Scan creates one line row per data row with raw fields.** `test_scan_creates_one_line_row_per_data_row_with_raw_fields`	1
160	**Scan stores mode and action on header and lines.** The first row has empty Mode/Action cells: the header takes the first non-empty value in the file, and that row falls back to it. `test_scan_stores_mode_and_action_on_header_and_lines`	1
161	**Scan without mode and action columns leaves them null.** `test_scan_without_mode_and_action_columns_leaves_them_null`	1
162	**Matched EPC is converted and enriched from tag management.** `test_matched_epc_is_converted_and_enriched_from_tag_management`	1
163	**Unmatched EPC is no match with null snapshot fields.** `test_unmatched_epc_is_no_match_with_null_snapshot_fields`	1
164	**Matched scanner is enriched from scanners registry.** `test_matched_scanner_is_enriched_from_scanners_registry`	1
165	**Unmatched scanner leaves snapshot fields null but keeps raw text.** `test_unmatched_scanner_leaves_snapshot_fields_null_but_keeps_raw_text`	1
166	**Scanner matching is case insensitive.** `test_scanner_matching_is_case_insensitive`	1
167	**Trailing blank line produces no extra row.** `test_trailing_blank_line_produces_no_extra_row`	1
168	**User without permission gets 403 (forbidden) on list and cancel.** `test_user_without_permission_gets_403_on_list_and_cancel`	1
169	**View only permission cannot cancel.** `test_view_only_permission_cannot_cancel`	1
170	**Cancelling an unknown line returns 404 (not found).** `test_cancelling_an_unknown_line_returns_404`	1
171	**Cancel sets status and is reflected in the list.** `test_cancel_sets_status_and_is_reflected_in_the_list`	1
172	**List line data includes header filename.** `test_list_line_data_includes_header_filename`	1
173	**Lines from one file are listed in the same order as the CSV.** A single scan commits every line from a file in one transaction, so they can all end up with the exact same created_at timestamp — ordering must rely on line_number, not created_at, to reproduce the CSV's own row order every time. `test_lines_from_one_file_are_listed_in_the_same_order_as_the_csv`	1
174	**Newest scanned file is listed first.** `test_newest_scanned_file_is_listed_first`	1
175	**Sync converts a line once its tag is registered afterwards.** `test_sync_converts_a_line_once_its_tag_is_registered_afterwards`	1
176	**Sync reflects a product rename on an already converted line.** `test_sync_reflects_a_product_rename_on_an_already_converted_line`	1
177	**Sync leaves cancelled lines untouched.** `test_sync_leaves_cancelled_lines_untouched`	1
178	**Sync reflects a scanner rename on an already matched line.** `test_sync_reflects_a_scanner_rename_on_an_already_matched_line`	1
179	**Sync reflects a changed scanner type.** `test_sync_reflects_a_changed_scanner_type`	1

180	**Sync leaves cancelled lines scanner snapshot untouched.** `test_sync_leaves_cancelled_lines_scanner_snapshot_untouched`	1
181	**Sync reports zero updates when nothing changed.** `test_sync_reports_zero_updates_when_nothing_changed`	1
182	**View only permission cannot sync.** `test_view_only_permission_cannot_sync`	1
183	**Sync matches a tag whose EPC uid has stray whitespace.** Simulates a row that predates the epc_uid-trimming validator (or was inserted outside the API) — a trailing tab, invisible in most UIs, that would otherwise make an identical EPC silently never match. `test_sync_matches_a_tag_whose_epc_uid_has_stray_whitespace`	1
184	**User without permission cannot sync.** `test_user_without_permission_cannot_sync`	1

`test_line_processing.py` 18

Tests for carrying out a scanned line's action (line_processing.py): every freshly scanned line and file starts "new"; the pending count/list only offer actions with a handler ("Assignment"); processing needs a product per file, creates the missing tags with it (or fills it in on a registered tag without one) and marks lines "loaded" with a comment; cancelling needs a comment; and a file's own status follows its ...

185	**Scan starts header and lines as new.** `test_scan_starts_header_and_lines_as_new`	1
186	**Pending only counts new assignment lines.** Case and surrounding spaces don't matter; other actions have no handler yet, so they're not offered. `test_pending_only_counts_new_assignment_lines`	1
187	**Book in lines are counted but left for StockMaster.** "Book In" lines are waiting work too, but StockMaster books them in — processing here never touches them. `test_book_in_lines_are_counted_but_left_for_stockmaster`	1
188	**Process creates missing tags with the files product.** `test_process_creates_missing_tags_with_the_files_product`	1
189	**Process requires a product.** `test_process_requires_a_product`	1
190	**Process only the given files.** `test_process_only_the_given_files`	1
191	**Products list for the dialog.** `test_products_list_for_the_dialog`	1
192	**Cancel requires a comment and updates line and header.** `test_cancel_requires_a_comment_and_updates_line_and_header`	1
193	**Cancel unknown line returns 404 (not found).** `test_cancel_unknown_line_returns_404`	1
194	**File without lines is loaded at scan time.** `test_file_without_lines_is_loaded_at_scan_time`	1
195	**Count needs only module access but list needs view.** `test_count_needs_only_module_access_but_list_needs_view`	1
196	**Process needs tag create permission too.** `test_process_needs_tag_create_permission_too`	1
197	**Deleting a tag matched by scanned lines unlinks them.** Regression: a tag referenced by Tag Linedata rows used to fail with a 500 (foreign key). Now its lines are unlinked first. `test_deleting_a_tag_matched_by_scanned_lines_unlinks_them`	1
198	**Bulk deleting tags matched by scanned lines unlinks them.** Same unlinking as the single delete above, for several tags at once. `test_bulk_deleting_tags_matched_by_scanned_lines_unlinks_them`	1
199	**Scan stores the CSV product comment column.** A column named "Comment" (any case): the first row's empty cell falls back to the file's first non-empty value. `test_scan_stores_the_csv_product_comment_column`	1

200	**Scan fills CSV comment cell from the comment column only.** "CSV Comment" reads only the column named "Comment": the unrelated last column is ignored, an empty cell stays empty (no fallback to the file), and a registered tag takes over its line's value. `test_scan_fills_csv_comment_cell_from_the_comment_column_only`	1
201	**Assignment copies CSV comment cell onto the new tag.** `test_assignment_copies_csv_comment_cell_onto_the_new_tag`	1
202	**Backfill fills CSV comment cell from the archived CSV.** `test_backfill_fills_csv_comment_cell_from_the_archived_csv`	1

`test_roles.py` 17

These tests check Tagscan's custom-roles-with-per-screen-permissions system: screens sync automatically, roles are fully custom (not a fixed enum), permissions are checked per screen per action, and a Tagscan admin can create/grant users scoped to Tagscan only — never any other module.

203	**Sync screens creates every defined screen.** `test_sync_screens_creates_every_defined_screen`	1
204	**Sync screens is safe to run more than once.** `test_sync_screens_is_safe_to_run_more_than_once`	1
205	**Sync screens updates the label of an existing screen.** `test_sync_screens_updates_the_label_of_an_existing_screen`	1
206	**User without module access cannot view dashboard.** `test_user_without_module_access_cannot_view_dashboard`	1
207	**User with access but no role cannot view roles screen.** `test_user_with_access_but_no_role_cannot_view_roles_screen`	1
208	**Role with view but not edit cannot rename a role.** `test_role_with_view_but_not_edit_cannot_rename_a_role`	1
209	**Super admin bypasses TagScan permissions entirely.** `test_super_admin_bypasses_tagscan_permissions_entirely`	1
210	**Creating a role with a duplicate name is rejected.** `test_creating_a_role_with_a_duplicate_name_is_rejected`	1
211	**Deleting a role still assigned to a user is rejected.** `test_deleting_a_role_still_assigned_to_a_user_is_rejected`	1
212	**Set role permissions replaces the whole matrix.** `test_set_role_permissions_replaces_the_whole_matrix`	1
213	**Permission on roles screen does not grant access to users screen.** `test_permission_on_roles_screen_does_not_grant_access_to_users_screen`	1
214	**Permission on users screen does not grant access to roles screen.** `test_permission_on_users_screen_does_not_grant_access_to_roles_screen`	1
215	**Cannot assign a role to a user without module access.** `test_cannot_assign_a_role_to_a_user_without_module_access`	1
216	**Create or grant user creates a brand new account scoped to TagScan only.** `test_create_or_grant_user_creates_a_brand_new_account_scoped_to_tagscan_only`	1
217	**Create or grant user without password for a new email is rejected.** `test_create_or_grant_user_without_password_for_a_new_email_is_rejected`	1
218	**Create or grant user with an existing email just grants access.** `test_create_or_grant_user_with_an_existing_email_just_grants_access`	1
219	**Me permissions reflects only what the role allows.** `test_me_permissions_reflects_only_what_the_role_allows`	1

`test_scanner_import.py` 5

These tests check the Scanners tile of TagScan's "Data Upload/Download" screen: template, XLSX import (create / update by name / row errors) and export, gated by the "tagscan.dataupload" screen permission.

220	**Template has the scanner columns.** `test_template_has_the_scanner_columns`	1
221	**Import creates updates and reports row errors.** `test_import_creates_updates_and_reports_row_errors`	1

222	**Export lists every scanner with its type name.** `test_export_lists_every_scanner_with_its_type_name`	1
223	**View only dataupload permission cannot import scanners.** `test_view_only_dataupload_permission_cannot_import_scanners`	1
224	**Import leaves columns missing from the file untouched.** `test_import_leaves_columns_missing_from_the_file_untouched`	1

`test_scanners.py` 24

These tests check the "Scanners" master-data screen: gated by normal module access + the "tagscan.scanners" permission, independently of "tagscan.tag-management"/others, with a super-admin bypass, full CRUD, and a delete guard while a scanner is still referenced by logged CSV data — mirrors test_tags.py's shape.

225	**User without module access cannot list scanners.** `test_user_without_module_access_cannot_list_scanners`	1
226	**User with access but no scanners permission cannot list scanners.** `test_user_with_access_but_no_scanners_permission_cannot_list_scanners`	1
227	**Scanners permission does not grant access to tag management screen.** `test_scanners_permission_does_not_grant_access_to_tag_management_screen`	1
228	**Super admin can create a scanner without an explicit role.** `test_super_admin_can_create_a_scanner_without_an_explicit_role`	1
229	**View only scanners permission cannot create a scanner.** `test_view_only_scanners_permission_cannot_create_a_scanner`	1
230	**View only scanners permission cannot update or delete.** `test_view_only_scanners_permission_cannot_update_or_delete`	1
231	**Create scanner with full field set.** `test_create_scanner_with_full_field_set`	1
232	**Creating a scanner strips whitespace from the name.** `test_creating_a_scanner_strips_whitespace_from_the_name`	1
233	**Creating a scanner with an invalid technology returns 422 (invalid input).** `test_creating_a_scanner_with_an_invalid_technology_returns_422`	1
234	**Creating a scanner without a type ID returns 422 (invalid input).** `test_creating_a_scanner_without_a_type_id_returns_422`	1
235	**Creating a scanner with an unknown type ID returns 404 (not found).** `test_creating_a_scanner_with_an_unknown_type_id_returns_404`	1
236	**Creating a scanner with a duplicate name is rejected.** `test_creating_a_scanner_with_a_duplicate_name_is_rejected`	1
237	**List scanners ordered by name.** `test_list_scanners_ordered_by_name`	1
238	**Super admin can update a scanner.** `test_super_admin_can_update_a_scanner`	1
239	**Updating a missing scanner returns 404 (not found).** `test_updating_a_missing_scanner_returns_404`	1
240	**Updating a scanner with an unknown type ID returns 404 (not found).** `test_updating_a_scanner_with_an_unknown_type_id_returns_404`	1
241	**Updating a scanner to a name colliding with another returns 409 (conflict).** `test_updating_a_scanner_to_a_name_colliding_with_another_returns_409`	1
242	**Super admin can delete an unused scanner.** `test_super_admin_can_delete_an_unused_scanner`	1
243	**Deleting a missing scanner returns 404 (not found).** `test_deleting_a_missing_scanner_returns_404`	1
244	**Generating an API key requires edit permission.** `test_generating_an_api_key_requires_edit_permission`	1
245	**Generating an API key returns a usable plaintext key once.** `test_generating_an_api_key_returns_a_usable_plaintext_key_once`	1
246	**Generating a new API key invalidates the previous one.** `test_generating_a_new_api_key_invalidates_the_previous_one`	1

247 **Revoking an API key blocks further uploads.** 1
`test_revoking_an_api_key_blocks_further_uploads`

248 **Deleting a scanner still referenced by a header row returns 400 (refused).** 1
`test_deleting_a_scanner_still_referenced_by_a_header_row_returns_400`

test_settings.py 5

Tests for TagScan's "Settings" screen: the DB-backed override of the receive-folder path (falling back to the .env-configured default), gated by the "tagscan.settings" permission independently of every other screen — see app/modules/module_1/router.py's get_settings/update_settings.

249 **User without settings permission cannot view settings.** 1
`test_user_without_settings_permission_cannot_view_settings`

250 **Get settings returns env default when nothing saved.** 1
`test_get_settings_returns_env_default_when_nothing_saved`

251 **View only permission cannot update settings.** 1
`test_view_only_permission_cannot_update_settings`

252 **Saving a new receive folder is reflected on get.** 1
`test_saving_a_new_receive_folder_is_reflected_on_get`

253 **Saved override is what the file browser and scan actually use.** 1
`test_saved_override_is_what_the_file_browser_and_scan_actually_use`

test_tags.py 31

These tests check the "TagManagement" screen: gated by normal module access + the "tagscan.tag-management" permission, independently of "tagscan.dashboard"/"roles"/"users", with a super-admin bypass and full CRUD — mirrors module_9's test_products.py shape.

254 **User without module access cannot list tags.** 1
`test_user_without_module_access_cannot_list_tags`

255 **User with access but no tag management permission cannot list tags.** 1
`test_user_with_access_but_no_tag_management_permission_cannot_list_tags`

256 **Tag management permission does not grant access to dashboard screen.** 1
`test_tag_management_permission_does_not_grant_access_to_dashboard_screen`

257 **Super admin can register a tag without an explicit role.** 1
`test_super_admin_can_register_a_tag_without_an_explicit_role`

258 **View only tag management permission cannot create a tag.** 1
`test_view_only_tag_management_permission_cannot_create_a_tag`

259 **Create tag with full field set.** 1
`test_create_tag_with_full_field_set`

260 **Creating a tag strips whitespace from the EPC uid.** 1
A stray leading/trailing space or tab (easy to pick up pasting from a spreadsheet) would otherwise make an identical EPC silently never match a scanned CSV line's own, already-stripped EPC.
`test_creating_a_tag_strips_whitespace_from_the_epc_uid`

261 **Creating a tag with an unknown assigned product ID returns 404 (not found).** 1
`test_creating_a_tag_with_an_unknown_assigned_product_id_returns_404`

262 **Creating a tag with a duplicate EPC uid is rejected.** 1
`test_creating_a_tag_with_a_duplicate_epc_uid_is_rejected`

263 **List tags ordered by EPC uid.** 1
`test_list_tags_ordered_by_epc_uid`

264 **Super admin can update a tag.** 1
`test_super_admin_can_update_a_tag`

265 **Updating a missing tag returns 404 (not found).** 1
`test_updating_a_missing_tag_returns_404`

266 **Super admin can delete a tag.** 1
`test_super_admin_can_delete_a_tag`

267 **Deleting a missing tag returns 404 (not found).** 1
`test_deleting_a_missing_tag_returns_404`

268	**Bulk delete removes only the given tags.** `test_bulk_delete_removes_only_the_given_tags`	1
269	**Bulk delete with an empty list is rejected.** `test_bulk_delete_with_an_empty_list_is_rejected`	1
270	**Bulk delete needs the delete permission.** `test_bulk_delete_needs_the_delete_permission`	1
271	**Downloading the template requires module access.** `test_downloading_the_template_requires_module_access`	1
272	**Super admin can download the template.** `test_super_admin_can_download_the_template`	1
273	**View only dataupload permission cannot import tags.** `test_view_only_dataupload_permission_cannot_import_tags`	1
274	**Import creates updates and reports row errors.** `test_import_creates_updates_and_reports_row_errors`	1
275	**Import accepts native excel date cells.** `test_import_accepts_native_excel_date_cells`	1
276	**Tag management permission alone cannot use the data upload tools.** `test_tag_management_permission_alone_cannot_use_the_data_upload_tools`	1
277	**Export round trips through the import.** `test_export_round_trips_through_the_import`	1
278	**Kars dropdown lists every kar by number.** `test_kars_dropdown_lists_every_kar_by_number`	1
279	**A tag can be linked to a kar and unlinked again.** `test_a_tag_can_be_linked_to_a_kar_and_unlinked_again`	1
280	**Linking a tag to an unknown kar returns 404 (not found).** `test_linking_a_tag_to_an_unknown_kar_returns_404`	1
281	**Deleting a kar unlinks its tags.** `test_deleting_a_kar_unlinks_its_tags`	1
282	**Kar nummer round trips through export and import.** `test_kar_nummer_round_trips_through_export_and_import`	1
283	**Export shows the CSV comment but the import ignores it.** `test_export_shows_the_csv_comment_but_the_import_ignores_it`	1
284	**Import leaves columns missing from the file untouched.** `test_import_leaves_columns_missing_from_the_file_untouched`	1

Wristband scans (shared) 14 tests

The wristband company's webhook and which counter sees a scan.

`test_wristband.py` 14

The wristband scans as a shared part: every message kept as a .json file and browsable in the Scans module, which counters show a scan (only teams with a reservation in the season, plus unmatched teams), the first counter to use a scan winning, and sending a test scan from Scans, Atelier and Walkies with each module's own right.

285	**Message is kept as a file and browsable.** `test_message_is_kept_as_a_file_and_browsable`	1
286	**Path outside the JSON folder is refused.** `test_path_outside_the_json_folder_is_refused`	1
287	**Failed file write never loses the scan.** The JSON "folder" is a file, so writing into it fails. `test_failed_file_write_never_loses_the_scan`	1
288	**Refused senders leave no file.** Anyone can post to the public address: without the token nothing lands on disk. `test_refused_senders_leave_no_file`	1
289	**Only teams with a reservation show at the atelier desk.** `test_only_teams_with_a_reservation_show_at_the_atelier_desk`	1

290 **A reservation at any festival of the season counts.** 1
terras' only package is at Rock Werchter; another festival doesn't change that.
`test_a_reservation_at_any_festival_of_the_season_counts`

291 **Without season the last open one counts.** 1
`test_without_season_the_last_open_one_counts`

292 **First counter to use a scan wins.** 1
`test_first_counter_to_use_a_scan_wins`

293 **Test scan from scans and walkies.** 1
`test_test_scan_from_scans_and_walkies`

294 **Each module needs its own right.** 1
`test_each_module_needs_its_own_right`

295 **Scanner label and settings in scans.** 1
`test_scanner_label_and_settings_in_scans`

296 **Scan filters.** 1
`test_scan_filters`

297 **Generated token is accepted replaced and revoked.** 1
`test_generated_token_is_accepted_replaced_and_revoked`

298 **Only edit on settings may change the token.** 1
`test_only_edit_on_settings_may_change_the_token`

KarTracker (module-2) 184 tests

Kars, locations, planning screens, karcode, PDFs and maps.

`test_dashboard.py` 8

These tests check KarTracker's landing dashboard ("KPI Overview"): aggregate KPI stats gated by plain module access (no specific screen permission), mirroring backend/tests/modules/module_3/test_dashboard.py. The movement windows are tested directly on build_dashboard_stats with a pinned "now", so they don't depend on the clock.

299 **Dashboard requires login.** 1
`test_dashboard_requires_login`

300 **Dashboard requires module access.** 1
`test_dashboard_requires_module_access`

301 **Dashboard is open to any module member and starts empty.** 1
`test_dashboard_is_open_to_any_module_member_and_starts_empty`

302 **Fleet figures count transport types and zero fill statuses.** 1
`test_fleet_figures_count_transport_types_and_zero_fill_statuses`

303 **Movement windows.** 1
`test_movement_windows`

304 **Plan kar coverage for a season.** 1
`test_plan_kar_coverage_for_a_season`

305 **Unknown season is 404 (not found).** 1
`test_unknown_season_is_404`

306 **Movements are grouped per belgian day.** 1
23:30 UTC on 30 June is already 01:30 on 1 July in Belgium (summer time).
`test_movements_are_grouped_per_belgian_day`

`test_delivery_order.py` 7

Tests for the "Leveringsvolgorde" screen (module_2/delivery_order_service.py): the active delivery locations get their position in the delivery tour (KarTracker_afleverlocaties.terrein_positie), saved as one whole tour.

307 **List puts the tour first and hides inactive.** 1
`test_list_puts_the_tour_first_and_hides_inactive`

308 **Save renumbers the tour and clears the rest.** 1
`test_save_renumbers_the_tour_and_clears_the_rest`

309 **Save rejects duplicate unknown or inactive IDs.** 1
`test_save_rejects_duplicate_unknown_or_inactive_ids`

310	**View right can read but not save.** `test_view_right_can_read_but_not_save`	1
311	**Without rights the screen is forbidden.** `test_without_rights_the_screen_is_forbidden`	1
312	**Edit right can save.** `test_edit_right_can_save`	1
313	**View right can load the ground plans for the map.** `test_view_right_can_load_the_ground_plans_for_the_map`	1

`test_groundplan.py` 7

Tests for KarTracker's ground plans (the /groundplans endpoints in module_2/router.py): several plans can be added, listed, updated (with or without a new image) and deleted, and the image upload is validated.

314	**Multiple groundplans can be created and listed.** `test_multiple_groundplans_can_be_created_and_listed`	1
315	**Groundplan rejects oversized image.** `test_groundplan_rejects_oversized_image`	1
316	**Groundplan rejects non image file.** `test_groundplan_rejects_non_image_file`	1
317	**Groundplan rejects inverted bounds.** `test_groundplan_rejects_inverted_bounds`	1
318	**Groundplan update without image keeps the old image.** `test_groundplan_update_without_image_keeps_the_old_image`	1
319	**Groundplan delete removes it.** `test_groundplan_delete_removes_it`	1
320	**Groundplan view only user cannot create.** `test_groundplan_view_only_user_cannot_create`	1

`test_help.py` 2

KarTracker's help manual endpoint (GET /help/pdf): module access is needed, and a topic of a screen the user can't view is refused — the general chapters (how it works, the overview, the phone) are open to everyone with access.

321	**Help needs module access.** `test_help_needs_module_access`	1
322	**Topics follow the role.** `test_topics_follow_the_role`	1

`test_initial_delivery.py` 6

Tests for the "Initiële leverplanning" screen (module_2/initial_delivery_service.py): per distributiepunt and delivery location, in tour order, the karren delivered there at their team's first festival of the season.

323	**Karren go to their team first festival location in tour order.** `test_karren_go_to_their_team_first_festival_location_in_tour_order`	1
324	**Team without plan for its first festival is left out.** `test_team_without_plan_for_its_first_festival_is_left_out`	1
325	**Needs the screen right and a known season.** `test_needs_the_screen_right_and_a_known_season`	1
326	**Print returns a landscape PDF with and without the map.** `test_print_returns_a_landscape_pdf_with_and_without_the_map`	1
327	**Print without a ground plan still draws the map.** `test_print_without_a_ground_plan_still_draws_the_map`	1
328	**Print needs known karren season and the screen right.** `test_print_needs_known_karren_season_and_the_screen_right`	1

`test_kar_actions.py` 9

Tests for the "Manuele kar beweging" screen (see module_2/router.py's /kar-actions endpoints): logging a kar movement into KarTracker_kar_actions, with the kar's team as snapshot, and updating the kar's own latest status/location at the same time.

329	**Logging a movement stores it and updates the kar.** `test_logging_a_movement_stores_it_and_updates_the_kar`	1

330	**Team is a snapshot of the kars team.** `test_team_is_a_snapshot_of_the_kars_team`	1
331	**Unknown kar or status is rejected.** `test_unknown_kar_or_status_is_rejected`	1
332	**Out of range coordinates are rejected.** `test_out_of_range_coordinates_are_rejected`	1
333	**History is newest first and filterable by kar.** `test_history_is_newest_first_and_filterable_by_kar`	1
334	**Lookups list karren with team and statuses.** `test_lookups_list_karren_with_team_and_statuses`	1
335	**Deleting a movement removes only the log.** `test_deleting_a_movement_removes_only_the_log`	1
336	**View only role can list but not log.** `test_view_only_role_can_list_but_not_log`	1
337	**User without actions permission cannot view history.** `test_user_without_actions_permission_cannot_view_history`	1

`test_kar_import.py` 8

These tests check KarTracker's "Data Upload/Download" screen: the XLSX template download, the bulk kar import (own screen key "kartracker.dataupload", separate from "kartracker.karmanagement"), and the full-database export. A kar_nummer that already exists is updated: filled-in cells overwrite, empty cells keep the saved value — see kar_import.py's own module docstring.

338	**Downloading the template requires dataupload permission.** `test_downloading_the_template_requires_dataupload_permission`	1
339	**Super admin can download the template.** `test_super_admin_can_download_the_template`	1
340	**Export contains karren and kar statuses sheets.** `test_export_contains_karren_and_kar_statuses_sheets`	1
341	**View only dataupload permission cannot import karren.** `test_view_only_dataupload_permission_cannot_import_karren`	1
342	**Import creates new karren updates existing and rejects bad lookups.** `test_import_creates_new_karren_updates_existing_and_rejects_bad_lookups`	1
343	**Kar code column is exported and imported.** `test_kar_code_column_is_exported_and_imported`	1
344	**Import update overwrites filled cells and keeps empty ones.** `test_import_update_overwrites_filled_cells_and_keeps_empty_ones`	1
345	**Import rejects a kar number repeated in the same file.** `test_import_rejects_a_kar_number_repeated_in_the_same_file`	1

`test_kar_management.py` 16

These tests check KarTracker's Karlijst phase: the KarStatussen lookup and the KarManagement fleet registry itself. Both are gated by their own screen keys ("kartracker.karstatuses"/"kartracker.karmanagement") through the same custom-roles system tested in test_roles.py, so here we focus on CRUD behaviour and the cross-module transport-type foreign key (into module-9's MasterData_product table) instead of re-proving ...

346	**Create and list kar statuses.** `test_create_and_list_kar_statuses`	1
347	**Creating a kar status with a duplicate name is rejected.** `test_creating_a_kar_status_with_a_duplicate_name_is_rejected`	1
348	**Deleting a kar status still used by a kar is rejected.** `test_deleting_a_kar_status_still_used_by_a_kar_is_rejected`	1
349	**Deleting an unused kar status succeeds.** `test_deleting_an_unused_kar_status_succeeds`	1
350	**Create kar succeeds with valid lookup IDs.** `test_create_kar_succeeds_with_valid_lookup_ids`	1
351	**Create kar with unknown team ID returns 404 (not found).** `test_create_kar_with_unknown_team_id_returns_404`	1

352	**Create kar without a team succeeds.** `test_create_kar_without_a_team_succeeds`	1
353	**Create kar with unknown status ID returns 404 (not found).** `test_create_kar_with_unknown_status_id_returns_404`	1
354	**Create kar with unknown transport type ID returns 404 (not found).** `test_create_kar_with_unknown_transport_type_id_returns_404`	1
355	**Creating a kar with a duplicate kar nummer is rejected.** `test_creating_a_kar_with_a_duplicate_kar_nummer_is_rejected`	1
356	**Update kar changes its fields.** `test_update_kar_changes_its_fields`	1
357	**Kar code is stored trimmed and kept when not sent.** `test_kar_code_is_stored_trimmed_and_kept_when_not_sent`	1
358	**Formula code for kar nummer uses the digits.** `test_formula_code_for_kar_nummer_uses_the_digits`	1
359	**Delete kar removes it.** `test_delete_kar_removes_it`	1
360	**User without karmanagement permission cannot view karren.** `test_user_without_karmanagement_permission_cannot_view_karren`	1
361	**User with karmanagement view only cannot create a kar.** `test_user_with_karmanagement_view_only_cannot_create_a_kar`	1

`test_kar_map.py` 2

Tests for the read-only Kar Map screen (see kartracker.py's KarMapResponse and module_2/router.py's list_kar_map): a combined Kar/Afleverlocatie/Distributiepunt payload, gated by its own "kartracker.karmap" permission, independent of the underlying masterdata screens' own view permissions.

362	**User without karmap permission cannot view it.** `test_user_without_karmap_permission_cannot_view_it`	1
363	**Kar map returns all three layers including rows without coordinates.** `test_kar_map_returns_all_three_layers_including_rows_without_coordinates`	1

`test_kar_moves.py` 5

Tests for the "Verplaatsingen tussen festivals" screen (module_2/kar_moves_service.py): for festival A -> B, the karren whose team's planned location changes, grouped per pick-up location in tour order, and its PDF.

364	**Only karren whose location changes are listed per pick up location.** `test_only_karren_whose_location_changes_are_listed_per_pick_up_location`	1
365	**Without a pair only the festivals are returned.** `test_without_a_pair_only_the_festivals_are_returned`	1
366	**Rejects the same or an unknown festival and needs the screen right.** `test_rejects_the_same_or_an_unknown_festival_and_needs_the_screen_right`	1
367	**Print returns a landscape PDF with and without the map.** `test_print_returns_a_landscape_pdf_with_and_without_the_map`	1
368	**Print needs known karren and two different festivals.** `test_print_needs_known_karren_and_two_different_festivals`	1

`test_kar_planning.py` 10

Tests for the read-only Kar Planning report (see kartracker.py's KarPlanningResponse and module_2/router.py's list_kar_planning): a join across KarManagement and its status/team/transport-type lookups, gated by its own "kartracker.karplanning" permission, independent of "kartracker.karmanagement" itself.

369	**User without karplanning permission cannot view it.** `test_user_without_karplanning_permission_cannot_view_it`	1
370	**Kar planning returns denormalized joined fields.** `test_kar_planning_returns_denormalized_joined_fields`	1
371	**Kar planning adds one afleverlocatie column per active festival of the season.** `test_kar_planning_adds_one_afleverlocatie_column_per_active_festival_of_the_season`	1
372	**Kar planning marks festivals the team unchecked in altsien select.** `test_kar_planning_marks_festivals_the_team_unchecked_in_altsien_select`	1

373	**Print returns one PDF page per selected kar.** `test_print_returns_one_pdf_page_per_selected_kar`	1
374	**Print rejects unknown season and empty selection.** `test_print_rejects_unknown_season_and_empty_selection`	1
375	**User without karplanning permission cannot print.** `test_user_without_karplanning_permission_cannot_print`	1
376	**Request URL prefills known fields and skips blanks.** `test_request_url_prefills_known_fields_and_skips_blanks`	1
377	**Karblad PDF handles many festivals and non latin1 text.** `test_karblad_pdf_handles_many_festivals_and_non_latin1_text`	1
378	**Karblad PDF prints the stored kar code.** `test_karblad_pdf_prints_the_stored_kar_code`	1

`test_kar_status_import.py` 5

These tests check the KarStatussen tile on KarTracker's "Data Upload/ Download" screen: the XLSX template download, the bulk kar-status import (same "kartracker.dataupload" screen key the Karren tile uses), and the kar-statuses-only export. A name that already exists (ignoring case) is reported as "unchanged" — see kar_status_import.py's own module docstring.

379	**Downloading the template requires dataupload permission.** `test_downloading_the_template_requires_dataupload_permission`	1
380	**Super admin can download the template.** `test_super_admin_can_download_the_template`	1
381	**Export contains kar statuses sheet.** `test_export_contains_kar_statuses_sheet`	1
382	**View only dataupload permission cannot import kar statuses.** `test_view_only_dataupload_permission_cannot_import_kar_statuses`	1
383	**Import creates new statuses and leaves existing ones unchanged.** `test_import_creates_new_statuses_and_leaves_existing_ones_unchanged`	1

`test_karcode.py` 34

Tests for the karcode: the formula evaluator (module_2/karcode_formula.py) and the Karcode Simulator + Instellingen endpoints (module_2/karcode_router.py).

384	**Formula computes the code.** Runs for: LEFT(kar*500+3118; 4)-36-2111, =left((kar*500)+3118;4)-36-2111, RIGHT(kar*500+3118; 3)-36-118, MID(kar*1000; 2; 3)-36-600, MOD(kar; 7)-36-1, ROUND(kar*0,8; 0)-36-29, ROUND(2,5; 0)-1-3, INT(kar/7)-36-5, 2^3+kar-1-9, kar/4-10-2.5, LEFT(kar*500; 2)+1-36-19, kar- 036 -36 `test_formula_computes_the_code`	×12
385	**Formula refuses bad input.** Runs for: __import__('os')-invalid_character, kar.real-invalid_syntax, LEFT(kar, 4)-invalid_syntax, foo + 1-unknown_name, SUM(kar)-unknown_function, LEFT()-wrong_arguments, 1/0-division_by_zero, MOD(kar; 0)-division_by_zero, 9^999-number_too_large, 10000000^64-number_too_large, LEFT(kar; 0)-empty_result, -empty_formula … `test_formula_refuses_bad_input`	×13
386	**Kar number must be a whole number.** Runs for: , K12, 3.5, 11111111111111111111 `test_kar_number_must_be_a_whole_number`	×4
387	**Simulator uses the default and then the saved formula.** `test_simulator_uses_the_default_and_then_the_saved_formula`	1
388	**Errors come back as translatable codes.** `test_errors_come_back_as_translatable_codes`	1
389	**Simulator needs its own screen right.** `test_simulator_needs_its_own_screen_right`	1
390	**Settings view can test but only edit can save.** `test_settings_view_can_test_but_only_edit_can_save`	1
391	**Settings edit right can save.** `test_settings_edit_right_can_save`	1

test_leverdata.py 10

Tests for the "Delivery Dates" screen (see module_2/router.py's /leverdata endpoints): per active festival of a season, one delivery date and one pick-up date, saved as an upsert on the festival into Festivals_leverdatum (at most one row per festival).

392	**User without leverdata permission cannot use it.** `test_user_without_leverdata_permission_cannot_use_it`	1
393	**View only user can read but not save.** `test_view_only_user_can_read_but_not_save`	1
394	**Lists only active festivals of the season without dates.** `test_lists_only_active_festivals_of_the_season_without_dates`	1
395	**Unknown season is 404 (not found).** `test_unknown_season_is_404`	1
396	**Save inserts then second save updates without duplicating.** `test_save_inserts_then_second_save_updates_without_duplicating`	1
397	**Save allows only one of the two dates.** `test_save_allows_only_one_of_the_two_dates`	1
398	**Clearing both dates removes the record.** `test_clearing_both_dates_removes_the_record`	1
399	**Pickup before delivery is rejected.** `test_pickup_before_delivery_is_rejected`	1
400	**Invalid festivals are rejected.** `test_invalid_festivals_are_rejected`	1
401	**Deleting a festival removes its dates.** `test_deleting_a_festival_removes_its_dates`	1

test_leverdatum_import.py 5

These tests check the Leverdata tile on KarTracker's "Data Upload/ Download" screen: the pre-filled XLSX template, the bulk import (an upsert per festival, rows without dates skipped) and the export — all behind the same "kartracker.dataupload" screen key as the other tiles.

402	**Template lists active festivals with empty dates.** `test_template_lists_active_festivals_with_empty_dates`	1
403	**Export contains saved dates.** `test_export_contains_saved_dates`	1
404	**View only dataupload permission cannot import.** `test_view_only_dataupload_permission_cannot_import`	1
405	**Import creates updates skips and reports errors.** `test_import_creates_updates_skips_and_reports_errors`	1
406	**Import rejects bad dates and pickup before delivery.** `test_import_rejects_bad_dates_and_pickup_before_delivery`	1

test_master_data_imports.py 4

These tests check the "update instead of reject" rule of the Zones, Distributiepunten and Afleverlocaties tiles on KarTracker's "Data Upload/ Download" screen. They call the import functions directly (the endpoints around them only add the "kartracker.dataupload" permission check, which test_kar_import.py already covers):

407	**Zone import leaves an existing name unchanged.** `test_zone_import_leaves_an_existing_name_unchanged`	1
408	**Distributiepunt import updates filled cells and keeps empty ones.** `test_distributiepunt_import_updates_filled_cells_and_keeps_empty_ones`	1
409	**Afleverlocatie import updates and keeps active when blank.** `test_afleverlocatie_import_updates_and_keeps_active_when_blank`	1
410	**Afleverlocatie import reads terrein positie as a whole number.** `test_afleverlocatie_import_reads_terrein_positie_as_a_whole_number`	1

test_plan_kar.py 16

Tests for the "Plan a kar" screen (see module_2/router.py's /plan-kar endpoints): per team, one delivery location per active festival

of a season, saved as an upsert on (season, festival, team) into KarTracker_kar_afleverlocaties.

411	**User without plankar permission cannot view it.** `test_user_without_plankar_permission_cannot_view_it`	1
412	**Dropdown lists only include active entries.** `test_dropdown_lists_only_include_active_entries`	1
413	**Afleverlocatie dropdown includes the description.** `test_afleverlocatie_dropdown_includes_the_description`	1
414	**Matrix lists only active festivals of the season without locations.** `test_matrix_lists_only_active_festivals_of_the_season_without_locations`	1
415	**Matrix unknown season or team is 404 (not found).** `test_matrix_unknown_season_or_team_is_404`	1
416	**Save inserts then second save updates without duplicating.** `test_save_inserts_then_second_save_updates_without_duplicating`	1
417	**Assignments are kept per team.** `test_assignments_are_kept_per_team`	1
418	**Clearing a row removes its record.** `test_clearing_a_row_removes_its_record`	1
419	**Save rejects invalid references.** `test_save_rejects_invalid_references`	1
420	**Afleverlocatie in use by a plan cannot be deleted.** `test_afleverlocatie_in_use_by_a_plan_cannot_be_deleted`	1
421	**Afleverlocatie active flag round trips.** `test_afleverlocatie_active_flag_round_trips`	1
422	**Afleverlocatie dropdown includes the coordinates.** `test_afleverlocatie_dropdown_includes_the_coordinates`	1
423	**Plankar viewer can read the ground plans.** `test_plankar_viewer_can_read_the_ground_plans`	1
424	**Matrix lists only the festivals the team picked.** `test_matrix_lists_only_the_festivals_the_team_picked`	1
425	**Overview counts planned festivals per kar.** `test_overview_counts_planned_festivals_per_kar`	1
426	**Overview needs the plankar screen and a known season.** `test_overview_needs_the_plankar_screen_and_a_known_season`	1

`test_roles.py` 20

These tests check KarTracker's custom-roles-with-per-screen-permissions system: screens sync automatically, roles are fully custom (not a fixed enum), permissions are checked per screen per action, and a KarTracker admin can create/grant users scoped to KarTracker only — never any other module. This is the module's first development phase, so only the "kartracker.roles"/"kartracker.users" screens exist yet.

427	**Sync screens creates every defined screen.** `test_sync_screens_creates_every_defined_screen`	1
428	**Sync screens is safe to run more than once.** `test_sync_screens_is_safe_to_run_more_than_once`	1
429	**Sync screens updates the label of an existing screen.** `test_sync_screens_updates_the_label_of_an_existing_screen`	1
430	**User without module access cannot view roles.** `test_user_without_module_access_cannot_view_roles`	1
431	**User with access but no role cannot view roles screen.** `test_user_with_access_but_no_role_cannot_view_roles_screen`	1
432	**Role with view but not edit cannot rename a role.** `test_role_with_view_but_not_edit_cannot_rename_a_role`	1
433	**Super admin bypasses KarTracker permissions entirely.** `test_super_admin_bypasses_kartracker_permissions_entirely`	1
434	**Creating a role with a duplicate name is rejected.** `test_creating_a_role_with_a_duplicate_name_is_rejected`	1

435	**Deleting a role still assigned to a user is rejected.** `test_deleting_a_role_still_assigned_to_a_user_is_rejected`	1
436	**Set role permissions replaces the whole matrix.** `test_set_role_permissions_replaces_the_whole_matrix`	1
437	**Set role permissions removes permissions for an omitted screen.** `test_set_role_permissions_removes_permissions_for_an_omitted_screen`	1
438	**Set role permissions with unknown screen ID returns 404 (not found).** `test_set_role_permissions_with_unknown_screen_id_returns_404`	1
439	**Permission on roles screen does not grant access to users screen.** `test_permission_on_roles_screen_does_not_grant_access_to_users_screen`	1
440	**Permission on users screen does not grant access to roles screen.** `test_permission_on_users_screen_does_not_grant_access_to_roles_screen`	1
441	**Cannot assign a role to a user without module access.** `test_cannot_assign_a_role_to_a_user_without_module_access`	1
442	**Create or grant user creates a brand new account scoped to KarTracker only.** `test_create_or_grant_user_creates_a_brand_new_account_scoped_to_kartracker_only`	1
443	**Create or grant user without password for a new email is rejected.** `test_create_or_grant_user_without_password_for_a_new_email_is_rejected`	1
444	**Create or grant user with an existing email just grants access.** `test_create_or_grant_user_with_an_existing_email_just_grants_access`	1
445	**Me permissions reflects only what the role allows.** `test_me_permissions_reflects_only_what_the_role_allows`	1
446	**Me permissions creatable keys reflects create permission.** `test_me_permissions_creatable_keys_reflects_create_permission`	1

`test_seasons.py` 2

Tests for Kar Planning's own season dropdown endpoint (module_2/router.py list_open_seasons): only open seasons, newest first, gated by the Kar Planning screen — so its users don't need MasterData to pick a season.

447	**Seasons lists only open seasons newest first.** `test_seasons_lists_only_open_seasons_newest_first`	1
448	**Seasons requires the kar planning screen.** `test_seasons_requires_the_kar_planning_screen`	1

`test_special_tour_planning.py` 4

Tests for the "Speciale levertoeren-planning" screen (module_2/special_tour_planning_service.py): for a festival + tours, the karren of the tours' teams at their planned location for that festival, grouped per distributiepunt / location in tour order, and its PDF.

449	**Without festival or tours only the filters come back.** `test_without_festival_or_tours_only_the_filters_come_back`	1
450	**Karren of the tours teams at their location for the festival.** `test_karren_of_the_tours_teams_at_their_location_for_the_festival`	1
451	**Unknown festival tour season and screen right.** `test_unknown_festival_tour_season_and_screen_right`	1
452	**Print returns a landscape PDF with and without the map.** `test_print_returns_a_landscape_pdf_with_and_without_the_map`	1

`test_special_tours.py` 4

Tests for KarTracker's "Speciale levertoeren" masterdata screen (/special-tours): CRUD, unique names, no delete while a team is linked, the linked teams' names and the screen rights.

453	**Create list update and delete a tour.** `test_create_list_update_and_delete_a_tour`	1
454	**Names are unique.** `test_names_are_unique`	1
455	**Linked teams are listed and block the delete.** `test_linked_teams_are_listed_and_block_the_delete`	1

456 **Needs the screen rights.** 1
`test_needs_the_screen_rights`

Interventies (module-3) 43 tests

Intervention requests: staff, public QR form, phone, mails.

test_dashboard.py 4

These tests check Intervention Requests' landing dashboard ("KPI overview"): aggregate KPI stats gated by plain module access (no specific screen permission), mirroring backend/tests/modules/module_9/test_dashboard.py.

457 **Dashboard requires module access.** 1
`test_dashboard_requires_module_access`

458 **Dashboard with no data returns all zero stats.** 1
`test_dashboard_with_no_data_returns_all_zero_stats`

459 **Dashboard computes open closed and status breakdown.** 1
`test_dashboard_computes_open_closed_and_status_breakdown`

460 **Dashboard team breakdown merges real teams and free text names.** 1
`test_dashboard_team_breakdown_merges_real_teams_and_free_text_names`

test_mailing_list.py 12

Tests for Intervention Requests' "Mailing List" settings screen and the "new request" mail it drives (app/modules/module_3/notifications.py): every new request — staff screen (desktop or phone) or public QR form — mails the active addresses on the list, with the delivery-note PDF attached, and a mail problem never stops the request from being saved.

461 **Mailing list crud.** 1
`test_mailing_list_crud`

462 **Duplicate address is rejected.** 1
`test_duplicate_address_is_rejected`

463 **Invalid address is rejected.** 1
`test_invalid_address_is_rejected`

464 **Requests permission does not open the mailing list.** 1
`test_requests_permission_does_not_open_the_mailing_list`

465 **Staff created request mails active recipients with PDF.** 1
`test_staff_created_request_mails_active_recipients_with_pdf`

466 **Public form request also mails.** 1
`test_public_form_request_also_mails`

467 **Request by a plain requests role also mails.** 1
The phone section (/m) creates requests through this same endpoint, as a user who usually only has the requests screen's create right.
`test_request_by_a_plain_requests_role_also_mails`

468 **Editing a request does not mail.** 1
`test_editing_a_request_does_not_mail`

469 **No active recipients means no mail.** 1
`test_no_active_recipients_means_no_mail`

470 **A failing mail never blocks the request.** 1
`test_a_failing_mail_never_blocks_the_request`

471 **Send email is skipped without configuration.** 1
`test_send_email_is_skipped_without_configuration`

472 **Send email chunks recipients and swallows errors.** 1
`test_send_email_chunks_recipients_and_swallows_errors`

test_public_intervention_requests.py 8

These tests check the public (no-login) intervention-request endpoints in app/modules/module_3/public_router.py — reachable by customers via a QR code, with no authentication at all. Contrast with test_roles.py, which covers the staff-only, login-gated router.py.

473 **List public teams requires no auth.** 1
`test_list_public_teams_requires_no_auth`

474	**Create public request with existing team.** `test_create_public_request_with_existing_team`	1
475	**Create public request with free text team name.** `test_create_public_request_with_free_text_team_name`	1
476	**Create public request rejects both team ID and team name.** `test_create_public_request_rejects_both_team_id_and_team_name`	1
477	**Create public request rejects neither team ID nor team name.** `test_create_public_request_rejects_neither_team_id_nor_team_name`	1
478	**Create public request ignores internal only fields.** status_id/handled_by/team_cart_user_id aren't accepted by the public schema at all — sending them is simply ignored by FastAPI/Pydantic rather than trusted, unlike the staff-only create endpoint. `test_create_public_request_ignores_internal_only_fields`	1
479	**Create public request 404s for unknown team ID.** `test_create_public_request_404s_for_unknown_team_id`	1
480	**Create public request fails without default status configured.** `test_create_public_request_fails_without_default_status_configured`	1

`test_request_form_endpoints.py` 4

Tests for the endpoints the phone's Interventie Aanvragen screens rely on: the full per-action /me/permissions, the one-call /lookups for the request form, and fetching one request by id. All gated by the requests screen's own permissions, like the rest of the "Akties" screen.

481	**Me permissions lists create edit and delete separately.** `test_me_permissions_lists_create_edit_and_delete_separately`	1
482	**Lookups returns statuses teams and teamkar members.** `test_lookups_returns_statuses_teams_and_teamkar_members`	1
483	**Lookups requires the requests screen.** `test_lookups_requires_the_requests_screen`	1
484	**Get one request by ID.** `test_get_one_request_by_id`	1

`test_roles.py` 15

These tests check Intervention Requests' custom-roles-with-per-screen- permissions system — the same system TagScan/MasterData use (see tests/modules/module_9/test_roles.py), applied to module-3. Intervention Requests'/Statuses' own CRUD tests live in test_intervention_requests.py/ test_intervention_statuses.py alongside this file.

485	**Sync screens creates every defined screen.** `test_sync_screens_creates_every_defined_screen`	1
486	**Sync screens is safe to run more than once.** `test_sync_screens_is_safe_to_run_more_than_once`	1
487	**User without module access cannot view roles.** `test_user_without_module_access_cannot_view_roles`	1
488	**User with access but no role cannot view roles screen.** `test_user_with_access_but_no_role_cannot_view_roles_screen`	1
489	**Role with view but not edit cannot rename a role.** `test_role_with_view_but_not_edit_cannot_rename_a_role`	1
490	**Super admin bypasses intervention requests permissions entirely.** `test_super_admin_bypasses_intervention_requests_permissions_entirely`	1
491	**Creating a role with a duplicate name is rejected.** `test_creating_a_role_with_a_duplicate_name_is_rejected`	1
492	**Deleting a role still assigned to a user is rejected.** `test_deleting_a_role_still_assigned_to_a_user_is_rejected`	1
493	**Set role permissions replaces the whole matrix.** `test_set_role_permissions_replaces_the_whole_matrix`	1
494	**Permission on roles screen does not grant access to users screen.** `test_permission_on_roles_screen_does_not_grant_access_to_users_screen`	1

495 **Permission on users screen does not grant access to roles screen.** 1
`test_permission_on_users_screen_does_not_grant_access_to_roles_screen`

496 **Cannot assign a role to a user without module access.** 1
`test_cannot_assign_a_role_to_a_user_without_module_access`

497 **Create or grant user creates a brand new account scoped to intervention requests only.** 1
`test_create_or_grant_user_creates_a_brand_new_account_scoped_to_intervention_requests_only`

498 **Me permissions reflects only what the role allows.** 1
`test_me_permissions_reflects_only_what_the_role_allows`

499 **User without requests screen permission cannot list intervention requests.** 1
`test_user_without_requests_screen_permission_cannot_list_intervention_requests`

StockMaster (module-4) 123 tests

Warehouse stock, kars, bookings, kar needs, products, voice count.

`test_book_in_import.py` 6

Inboeken from Excel (app/modules/module_4/booking_import.py): the template lists the bookable products, and an uploaded copy becomes basket lines plus per-row problems — without booking anything.

500 **Template lists every bookable product.** 1
`test_template_lists_every_bookable_product`

501 **Upload returns basket lines and row problems.** 1
`test_upload_returns_basket_lines_and_row_problems`

502 **Ambiguous name is a row problem.** 1
`test_ambiguous_name_is_a_row_problem`

503 **Unreadable file is refused.** 1
`test_unreadable_file_is_refused`

504 **Import needs the book in right.** 1
`test_import_needs_the_book_in_right`

505 **Requirements editors may use the import.** 1
"Benodigdheden" fills a kar's needs list from the same template.
`test_requirements_editors_may_use_the_import`

`test_delivery_entitlement.py` 8

The flow diagram behind Kar laden's "Op dit moment beschikbaar voor levering" (app/modules/module_4/delivery_entitlement.py): each branch on its own, and the kar detail that fills `entitled` + the trace on every row.

506 **Limit names map to rules.** 1
`test_limit_names_map_to_rules`

507 **Blocked and logistics come first.** 1
`test_blocked_and_logistics_come_first`

508 **Unlimited gives what was asked.** 1
`test_unlimited_gives_what_was_asked`

509 **Automatic enough stock gives what was asked without cap.** 1
`test_automatic_enough_stock_gives_what_was_asked_without_cap`

510 **Automatic short gives a rounded down share capped until delivered.** 1
7 asked, 60 in stock for 100 asked in all: 60 % of 7 = 4.2 → 4.
`test_automatic_short_gives_a_rounded_down_share_capped_until_delivered`

511 **Percentage rounds down and caps.** 1
`test_percentage_rounds_down_and_caps`

512 **Assigned caps until delivered and shares the cap.** 1
`test_assigned_caps_until_delivered_and_shares_the_cap`

513 **Kar detail fills entitlement per row.** 1
Tape: Automatisch, 12 in stock; 10 needed on K101 and 10 on K043, 4 asked by hand on K101 → 24 asked.
`test_kar_detail_fills_entitlement_per_row`

test_help.py 3

StockMaster's help manual endpoint (GET /help/pdf): module access is needed, and a topic of a screen the user can't view is refused.

514 **Full manual and one topic.** 1
`test_full_manual_and_one_topic`

515 **Help needs module access.** 1
`test_help_needs_module_access`

516 **Topic of a hidden screen is not found.** 1
`test_topic_of_a_hidden_screen_is_not_found`

test_manual_requests.py 4

Kar laden's manual requests (app/modules/module_4/manual_request_service.py): products added with "Manueel product toevoegen" are stored per season and kar, add up, can be removed, follow the Kar laden right and the season lock, and stay after the product is loaded.

517 **Manual requests add up and can be removed.** 1
`test_manual_requests_add_up_and_can_be_removed`

518 **Manual request stays after loading.** 1
`test_manual_request_stays_after_loading`

519 **Manual request rules.** 1
A blocked product can't be asked for, and the quantity must be at least 1.
`test_manual_request_rules`

520 **Manual requests need the kar laden right.** 1
`test_manual_requests_need_the_kar_laden_right`

test_product_data_upload.py 9

These tests check the product half of StockMaster's "Data Upload/Download" screen (moved here from MasterData together with the product screens): a single "stockmaster.dataupload" screen key gates the XLSX template download, bulk import and export of products and their four lookup lists.

521 **Product import resolves lookups and booleans.** 1
`test_product_import_resolves_lookups_and_booleans`

522 **Product import rejects missing lookups percentage without value and huge cost.** 1
`test_product_import_rejects_missing_lookups_percentage_without_value_and_huge_cost`

523 **Product import rejects an unrecognized boolean value.** 1
`test_product_import_rejects_an_unrecognized_boolean_value`

524 **Product import allows duplicate names.** 1
`test_product_import_allows_duplicate_names`

525 **Product template needs dataupload view not the products screen.** 1
`test_product_template_needs_dataupload_view_not_the_products_screen`

526 **View only dataupload permission cannot import warehouses.** 1
`test_view_only_dataupload_permission_cannot_import_warehouses`

527 **Product export contains every product.** 1
`test_product_export_contains_every_product`

528 **Lookup row with an ID renames and unknown ID is an error.** 1
`test_lookup_row_with_an_id_renames_and_unknown_id_is_an_error`

529 **Product export reuploaded updates instead of duplicating.** 1
`test_product_export_reuploaded_updates_instead_of_duplicating`

test_product_lookups.py 28

These tests check the four product lookup screens — Type, Magazijn (Warehouse), Categorie (ProductCategory), and Limiet (ProductLimit) — nested under Products. All four share the exact same bare id+unique-name shape and endpoint behavior as MasterData's Season, so their CRUD + permission-gating behavior is tested once via parametrize instead of copy-pasted four times; what's specific to each — how Product actually ...

530 **User without permission cannot list.** ×4
Runs for: product-types-stockmaster.product-types, warehouses-stockmaster.warehouses, product-categories-stockmaster.product-categories, product-limits-stockmaster.product-limits
`test_user_without_permission_cannot_list`

531 **Super admin can create list rename and delete.** ×4
Runs for: product-types-stockmaster.product-types, warehouses-stockmaster.warehouses, product-categories-stockmaster.product-categories, product-limits-stockmaster.product-limits
`test_super_admin_can_create_list_rename_and_delete`

532 **View only permission cannot create.** ×4
Runs for: product-types-stockmaster.product-types, warehouses-stockmaster.warehouses, product-categories-stockmaster.product-categories, product-limits-stockmaster.product-limits
`test_view_only_permission_cannot_create`

533 **Creating a duplicate name is rejected.** ×4
Runs for: product-types-stockmaster.product-types, warehouses-stockmaster.warehouses, product-categories-stockmaster.product-categories, product-limits-stockmaster.product-limits
`test_creating_a_duplicate_name_is_rejected`

534 **Renaming or deleting a missing entity returns 404 (not found).** ×4
Runs for: product-types-stockmaster.product-types, warehouses-stockmaster.warehouses, product-categories-stockmaster.product-categories, product-limits-stockmaster.product-limits
`test_renaming_or_deleting_a_missing_entity_returns_404`

535 **Creating a product with an unknown lookup ID returns 404 (not found).** ×4
Runs for: product-types-type_id-ProductType, warehouses-warehouse_id-Warehouse, product-categories-category_id-ProductCategory, product-limits-limit_id-ProductLimit
`test_creating_a_product_with_an_unknown_lookup_id_returns_404`

536 **Deleting a lookup value still used by a product is rejected.** ×4
Runs for: product-types-type_id-ProductType, warehouses-warehouse_id-Warehouse, product-categories-category_id-ProductCategory, product-limits-limit_id-ProductLimit
`test_deleting_a_lookup_value_still_used_by_a_product_is_rejected`

`test_product_stats.py` 4

These tests check the product statistics on StockMaster's KPI page (GET /product-stats, moved here from MasterData's dashboard): gated by the "stockmaster.kpi" screen, all products (not per season).

537 **Product stats need the KPI screen.** 1
`test_product_stats_need_the_kpi_screen`

538 **Product stats with no data are all zero.** 1
`test_product_stats_with_no_data_are_all_zero`

539 **Product stats compute the tile counts.** 1
`test_product_stats_compute_the_tile_counts`

540 **Product stats breakdowns are zero filled and ordered with unassigned last.** 1
`test_product_stats_breakdowns_are_zero_filled_and_ordered_with_unassigned_last`

`test_products.py` 20

These tests check StockMaster's "Products" screen (moved here from MasterData): gated by normal module access + the "stockmaster.products" permission, independently of the other StockMaster screens, with a super-admin bypass and full CRUD.

541 **User without module access cannot list products.** 1
`test_user_without_module_access_cannot_list_products`

542 **User with access but no products permission cannot list products.** 1
`test_user_with_access_but_no_products_permission_cannot_list_products`

543 **Products permission does not grant access to roles screen.** 1
`test_products_permission_does_not_grant_access_to_roles_screen`

544 **Super admin can manage products without an explicit role.** 1
`test_super_admin_can_manage_products_without_an_explicit_role`

545 **View only products permission cannot create a product.** 1
`test_view_only_products_permission_cannot_create_a_product`

546 **Create product with full field set.** 1
`test_create_product_with_full_field_set`

547 **Type warehouse category and stock return are required.** 1
`test_type_warehouse_category_and_stock_return_are_required`

548 **Product cost must not be negative and can be cleared.** 1
`test_product_cost_must_not_be_negative_and_can_be_cleared`

549	**List products ordered by name.** `test_list_products_ordered_by_name`	1
550	**Super admin can update a product.** `test_super_admin_can_update_a_product`	1
551	**Updating a missing product returns 404 (not found).** `test_updating_a_missing_product_returns_404`	1
552	**Super admin can delete a product.** `test_super_admin_can_delete_a_product`	1
553	**Deleting a missing product returns 404 (not found).** `test_deleting_a_missing_product_returns_404`	1
554	**Upload and read back a product image.** `test_upload_and_read_back_a_product_image`	1
555	**Product image must be an image.** `test_product_image_must_be_an_image`	1
556	**Product image must be smaller than 5 mb.** `test_product_image_must_be_smaller_than_5_mb`	1
557	**Delete product image keeps the product.** `test_delete_product_image_keeps_the_product`	1
558	**Create only role can attach an image but view only cannot.** `test_create_only_role_can_attach_an_image_but_view_only_cannot`	1
559	**Product used by a kar or RFID tag cannot be deleted.** `test_product_used_by_a_kar_or_rfid_tag_cannot_be_deleted`	1
560	**Limit is required and percentage needs a limit value.** `test_limit_is_required_and_percentage_needs_a_limit_value`	1

`test_roles.py` 6

StockMaster's access rights: the same custom-roles-with-per-screen- permissions system as the other modules (see tests/modules/module_8/ test_roles.py) — module access first, then view/create/edit/delete per screen.

561	**User without module access is refused.** `test_user_without_module_access_is_refused`	1
562	**Super admin without module access is refused.** Being super admin doesn't open a module by itself (same rule as every module). `test_super_admin_without_module_access_is_refused`	1
563	**My permissions follow the role.** `test_my_permissions_follow_the_role`	1
564	**Booking needs the matching create right.** Viewing the stock isn't enough to book in. `test_booking_needs_the_matching_create_right`	1
565	**Order needs create right may book in.** `test_order_needs_create_right_may_book_in`	1
566	**Roles and users screens.** `test_roles_and_users_screens`	1

`test_stock.py` 23

StockMaster's booking rules (app/modules/module_4/stock_service.py): free stock and kar stock side by side, never negative, kar departures and returns, counts, undo, the season lock — and every booking stored under the logged-in user.

567	**Product in free stock and several kars.** `test_product_in_free_stock_and_several_kars`	1
568	**Stock can never go negative.** `test_stock_can_never_go_negative`	1
569	**Booking is stored under the logged in user.** `test_booking_is_stored_under_the_logged_in_user`	1
570	**Kar to kar move.** `test_kar_to_kar_move`	1

571	**Book out needs a reason and works from a kar.** `test_book_out_needs_a_reason_and_works_from_a_kar`	1
572	**Blocked product cannot be booked in.** `test_blocked_product_cannot_be_booked_in`	1
573	**Dispatch books the contents out and return books back in.** `test_dispatch_books_the_contents_out_and_return_books_back_in`	1
574	**Loading a kar that is out is a delivery to the festival.** `test_loading_a_kar_that_is_out_is_a_delivery_to_the_festival`	1
575	**Delivery to an out kar must be in the season of its trip.** `test_delivery_to_an_out_kar_must_be_in_the_season_of_its_trip`	1
576	**Kar detail shows what was consumed outside the warehouse.** `test_kar_detail_shows_what_was_consumed_outside_the_warehouse`	1
577	**Count books only the differences.** `test_count_books_only_the_differences`	1
578	**Undo books the opposite once.** `test_undo_books_the_opposite_once`	1
579	**Undo of a departure brings the contents back.** `test_undo_of_a_departure_brings_the_contents_back`	1
580	**Closed season is locked.** `test_closed_season_is_locked`	1
581	**Minimum stock alert for consumables.** `test_minimum_stock_alert_for_consumables`	1
582	**Requirements drive progress and order needs.** `test_requirements_drive_progress_and_order_needs`	1
583	**Requirements change needs the matching right.** `test_requirements_change_needs_the_matching_right`	1
584	**Count sheet and history filters.** `test_count_sheet_and_history_filters`	1
585	**Every PDF renders.** `test_every_pdf_renders`	1
586	**Load lists PDF needs known kars and the kars right.** `test_load_lists_pdf_needs_known_kars_and_the_kars_right`	1
587	**Product and kar with stock cannot be deleted.** `test_product_and_kar_with_stock_cannot_be_deleted`	1
588	**Product picture is served to StockMaster users.** Without a picture: the stock line says so and the image is a 404. `test_product_picture_is_served_to_stockmaster_users`	1
589	**Product picture needs module access.** `test_product_picture_needs_module_access`	1

`test_tagscan_book_in.py` 5

Inboeken from a TagScan scan (app/modules/module_4/tagscan_book_in.py): waiting "Book In" lines become basket lines per scanned file (one piece per distinct tag, of the tag's product), lines with an unknown tag or a tag without product are problems, and the booking marks the lines it came from "loaded" in the same transaction — never twice.

590	**Waiting lines become basket lines per file.** `test_waiting_lines_become_basket_lines_per_file`	1
591	**Booking marks the scan lines loaded.** `test_booking_marks_the_scan_lines_loaded`	1
592	**The same scan lines cant be booked twice.** `test_the_same_scan_lines_cant_be_booked_twice`	1
593	**Cancel a problem line with a reason.** `test_cancel_a_problem_line_with_a_reason`	1
594	**Count needs module access but list needs book in.** `test_count_needs_module_access_but_list_needs_book_in`	1

test_voice_counts.py 7

Tests for StockMaster's voice counts (the phone's "Spraaktelling"): saving a kar's count, merging a product said twice, the checks on kar/product/ quantity/season, the screen rights, cancelling and the CSV download.

595	**Lookups list kars and unblocked products.** `test_lookups_list_kars_and_unblocked_products`	1
596	**Save merges a product said twice.** `test_save_merges_a_product_said_twice`	1
597	**Save does not change stock.** `test_save_does_not_change_stock`	1
598	**Save refuses bad input.** `test_save_refuses_bad_input`	1
599	**Closed season is refused.** `test_closed_season_is_refused`	1
600	**List detail cancel and CSV.** `test_list_detail_cancel_and_csv`	1
601	**Rights follow the voice count screen.** Only "create" on the voice count screen: may count on the phone, not see the list. `test_rights_follow_the_voice_count_screen`	1

Atelier (module-5) 93 tests

Work clothing: master data, reservations, stock, counter desk, RFID, mails.

test_backdate.py 3

Back-dated Atelier bookings (`booked_at`), used to load last year's desk lists (backend/masstest): only a super admin may send one, never in the future; the booking, the pick-up and the return keep that moment (a time without a zone is Belgian time), the document number takes its Belgian year, the wash statistics follow it, and back-dated desk actions never mail.

602	**Only a super admin and never in the future.** `test_only_a_super_admin_and_never_in_the_future`	1
603	**Pick up and return keep their moment and send no mail.** 10:15 Belgian summer time = 08:15 UTC. `test_pick_up_and_return_keep_their_moment_and_send_no_mail`	1
604	**Document number year and wash stats follow the moment.** 00:30 on 1 January in Belgium is still 31 December in UTC: the Belgian year counts. `test_document_number_year_and_wash_stats_follow_the_moment`	1

test_desk.py 20

Atelier's counter desk (block 4): Ophalen (one piece per package article, size picked at the desk, available → handed out) and Terugbrengen (handed out → to wash, partial allowed), the rules (package only, first + family name = the person, email required but not unique, unique bracelet number, no second piece of an article while one is out), "Uitgifte annuleren", the open list, the persons-vs-reservation count, the …

605	**Context teams and pick up.** `test_context_teams_and_pick_up`	1
606	**Pick up rules.** `test_pick_up_rules`	1
607	**Partial and full return.** `test_partial_and_full_return`	1
608	**Cancel only while nothing returned.** `test_cancel_only_while_nothing_returned`	1
609	**KPI and rights.** `test_kpi_and_rights`	1
610	**Email not unique and bracelet number unique.** Two different people may share an email address. `test_email_not_unique_and_bracelet_number_unique`	1

611	**Pick up twice a day and only some articles.** Only the apron (one package article of two) is fine. `test_pick_up_twice_a_day_and_only_some_articles`	1
612	**Context shows team responsibles.** The team's leads of the festival's season are on the desk context; other seasons' and teams' are not. `test_context_shows_team_responsibles`	1
613	**Return status MasterData and issues.** The list: a name exists once (ignoring case). `test_return_status_masterdata_and_issues`	1
614	**Excel downloads.** `test_excel_downloads`	1
615	**All festivals of a season.** Openstaand and Retourproblemen with season_id ("Alle festivals"): every festival of that season, with its name; other seasons are left out. `test_all_festivals_of_a_season`	1
616	**A role with one balie screen can load the festivals.** Runs for: atelier.desk, atelier.open, atelier.returnissues `test_a_role_with_one_balie_screen_can_load_the_festivals`	×3
617	**Write off from openstaand.** `test_write_off_from_openstaand`	1
618	**Write off only own pieces and with the right.** `test_write_off_only_own_pieces_and_with_the_right`	1
619	**Other wristband at the festival needs a confirmation.** `test_other_wristband_at_the_festival_needs_a_confirmation`	1
620	**Return search finds people with pieces behind a common name.** Twelve people called Jan, of whom only the last has a piece out. `test_return_search_finds_people_with_pieces_behind_a_common_name`	1
621	**Deleting MasterData with hand outs is refused clearly.** `test_deleting_masterdata_with_hand_outs_is_refused_clearly`	1
622	**Reservation limit.** Terras reserved 2 persons: the people holding clothing count, not who came earlier. `test_reservation_limit`	1

`test_help.py` 3

Atelier's help manual endpoint (GET /help/pdf): module access is needed, and a topic of a screen the user can't view is refused.

623	**Full manual and one topic.** `test_full_manual_and_one_topic`	1
624	**Help needs module access.** `test_help_needs_module_access`	1
625	**Topic of a hidden screen is not found.** `test_topic_of_a_hidden_screen_is_not_found`	1

`test_mails.py` 6

Atelier's desk mails: after Ophalen and after Terugbrengen (the person-level return) the person gets the "Uitgifte" / "Inname" mail built from the admin's templates — none without an email or with the template switched off, one for a return spanning several pick-ups — and the "Instellingen > E-mails" endpoints (placeholder check, test mail, rights).

626	**Placeholders and rendering.** `test_placeholders_and_rendering`	1
627	**Mail after pick up and one mail per return.** `test_mail_after_pick_up_and_one_mail_per_return`	1
628	**Person return refuses someone elses piece.** `test_person_return_refuses_someone_elses_piece`	1
629	**No mail without email or when switched off.** `test_no_mail_without_email_or_when_switched_off`	1

630 **Template screen.** 1
Both templates with their default text, the placeholders and example values.
`test_template_screen`

631 **Template rights.** 1
View only: may read, not save or send a test.
`test_template_rights`

`test_mass_return.py` 3

Atelier's Massa-inname (the Raspberry Pi app): a team's box of clothing is read at once and every piece comes back, booked per person who had it out (one "Inname" mail each). One tag in the box that isn't a handed-out piece refuses the whole box. Its own right: atelier.massreturn "create".

632 **Mass return books per person.** 1
`test_mass_return_books_per_person`

633 **One wrong tag blocks the box.** 1
`test_one_wrong_tag_blocks_the_box`

634 **Mass return needs its own right.** 1
`test_mass_return_needs_its_own_right`

`test_masterdata.py` 10

Atelier's masterdata (block 1): sizes (name + fit), articles with their sizes, the per-season team packages (incl. "copy from last season"), the KPI counts and the XLSX import/export — plus the per-screen rights.

635 **Sizes are grouped by fit and unique per fit.** 1
`test_sizes_are_grouped_by_fit_and_unique_per_fit`

636 **Size used by an article cant be deleted.** 1
`test_size_used_by_an_article_cant_be_deleted`

637 **Article with sizes create update and unique code.** 1
`test_article_with_sizes_create_update_and_unique_code`

638 **Inactive size cant be added but stays on an article.** 1
`test_inactive_size_cant_be_added_but_stays_on_an_article`

639 **Article picture upload view and remove.** 1
`test_article_picture_upload_view_and_remove`

640 **MasterData stats.** 1
`test_masterdata_stats`

641 **Each screen needs its own right.** 1
A role that may only view Artikelen: it sees articles and sizes (for the form), but can't create, and can't open reservations or imports.
`test_each_screen_needs_its_own_right`

642 **XLSX imports and round trip.** 1
Sizes: Dutch fit names are accepted; an existing name + fit without id is updated (not duplicated); a bad fit is a row error.
`test_xlsx_imports_and_round_trip`

643 **XLSX import of another environments export.** 1
An export from another environment carries ids that don't exist here: those rows are matched by their natural key (code / name + fit) instead.
`test_xlsx_import_of_another_environments_export`

644 **Return statuses in data upload.** 1
`test_return_statuses_in_data_upload`

`test_reservations.py` 11

Atelier's reservations: per festival and per team the persons and the package (articles × pieces per person) on one screen; copy from the same festival last year; the preparation (persons × package, the same every festival day, sizes estimated with the article's split); the picklist PDF; the rights, the KPI line and the XLSX tiles; and the article delete rule.

645 **Split by shares always adds up.** 1
`test_split_by_shares_always_adds_up`

646 **Size split must add up to 100.** 1
`test_size_split_must_add_up_to_100`

647	**Every active team is listed.** Every active team, with or without festival picks or a package yet. `test_every_active_team_is_listed`	1
648	**Persons and package saved together.** `test_persons_and_package_saved_together`	1
649	**A reservation with a package and 0 persons stays.** `test_a_reservation_with_a_package_and_0_persons_stays`	1
650	**Copy from same festival last year.** `test_copy_from_same_festival_last_year`	1
651	**Preparation and picklist.** `test_preparation_and_picklist`	1
652	**KPI and rights.** `test_kpi_and_rights`	1
653	**Reservation and package line XLSX.** `test_reservation_and_package_line_xlsx`	1
654	**KPI shows teams forgotten at another festival.** `test_kpi_shows_teams_forgotten_at_another_festival`	1
655	**Festivals picked shows progress per team.** `test_festivals_picked_shows_progress_per_team`	1

`test_rfid.py` 9

Atelier's RFID pieces (the Raspberry Pi app): Tags koppelen (read tags become pieces of an article size), resolving read tags, Ophalen with a tag per piece (it must be exactly that size and not out) and Terugbrengen by the tags read (the open lines carry the EPC). Also the Pi's own way in: the same rights, through a Bearer token.

656	**Assign and resolve.** `test_assign_and_resolve`	1
657	**Hand out and return by tag.** `test_hand_out_and_return_by_tag`	1
658	**Hand out tag rules.** `test_hand_out_tag_rules`	1
659	**Rights and bearer token.** The fixtures logged in with every right through cookies: drop them, the Pi has none. `test_rights_and_bearer_token`	1
660	**TagScan shows the piece and keeps it apart.** A tag linked on the Pi shows its Atelier piece on TagScan's TagManagement. `test_tagscan_shows_the_piece_and_keeps_it_apart`	1
661	**Trace keeps the EPC and source.** `test_trace_keeps_the_epc_and_source`	1
662	**Tag history right.** `test_tag_history_right`	1
663	**Bookings show the pieces and EPC.** Boekingen: a desk booking lists its pieces with person and EPC, and its source. `test_bookings_show_the_pieces_and_epc`	1
664	**Stock counts pieces with and without tag.** The desk fixture received 3 pieces of T-shirt M and 5 aprons. `test_stock_counts_pieces_with_and_without_tag`	1

`test_roles.py` 14

These tests check Atelier's custom-roles-with-per-screen-permissions system — the same system Atelier uses (see tests/modules/module_5/test_roles.py), applied to module-5.

665	**Sync screens creates every defined screen.** `test_sync_screens_creates_every_defined_screen`	1
666	**Sync screens is safe to run more than once.** `test_sync_screens_is_safe_to_run_more_than_once`	1
667	**User without module access cannot view roles.** `test_user_without_module_access_cannot_view_roles`	1

668 **User with access but no role cannot view roles screen.** 1
`test_user_with_access_but_no_role_cannot_view_roles_screen`

669 **Role with view but not edit cannot rename a role.** 1
`test_role_with_view_but_not_edit_cannot_rename_a_role`

670 **Super admin bypasses atelier permissions entirely.** 1
`test_super_admin_bypasses_atelier_permissions_entirely`

671 **Creating a role with a duplicate name is rejected.** 1
`test_creating_a_role_with_a_duplicate_name_is_rejected`

672 **Deleting a role still assigned to a user is rejected.** 1
`test_deleting_a_role_still_assigned_to_a_user_is_rejected`

673 **Set role permissions replaces the whole matrix.** 1
`test_set_role_permissions_replaces_the_whole_matrix`

674 **Permission on roles screen does not grant access to users screen.** 1
`test_permission_on_roles_screen_does_not_grant_access_to_users_screen`

675 **Permission on users screen does not grant access to roles screen.** 1
`test_permission_on_users_screen_does_not_grant_access_to_roles_screen`

676 **Cannot assign a role to a user without module access.** 1
`test_cannot_assign_a_role_to_a_user_without_module_access`

677 **Create or grant user creates a brand new account scoped to atelier only.** 1
`test_create_or_grant_user_creates_a_brand_new_account_scoped_to_atelier_only`

678 **Me permissions reflects only what the role allows.** 1
`test_me_permissions_reflects_only_what_the_role_allows`

`test_scans.py` 6

Atelier's side of the wristband scans: the public webhook (token, logging of every message, other events, unreadable bodies), the auto-registered scanners, team matching, the desk's polling per scanner, using a scan for a pick-up / return, the wristband number per festival, the "wristband required" switch, the Test-scan endpoint and the rights.

679 **Webhook accepts logs and registers.** 1
`test_webhook_accepts_logs_and_registers`

680 **Webhook refuses and ignores but logs.** 1
`test_webhook_refuses_and_ignores_but_logs`

681 **No token configured refuses everything.** 1
`test_no_token_configured_refuses_everything`

682 **Scan used for pick up and return.** 1
`test_scan_used_for_pick_up_and_return`

683 **Wristband per festival and required switch.** 1
`test_wristband_per_festival_and_required_switch`

684 **Test scan endpoint and rights.** 1
No token needed; same processing; marked as a test.
`test_test_scan_endpoint_and_rights`

`test_stock.py` 6

Atelier's stock (block 3): receipts, washing, write-offs, counts, undo, the never-negative rule, the session user on bookings, the error codes, the rights, the shortage on Voorbereiden, Te bestellen, the minimum-stock warnings and the PDFs.

685 **Receipt wash write off and count.** 1
Ontvangst: 10 M + 4 L become available; the booker is the session user.
`test_receipt_wash_write_off_and_count`

686 **Undo by counter booking.** 1
`test_undo_by_counter_booking`

687 **Inactive size cant be received and history protects it.** 1
`test_inactive_size_cant_be_received_and_history_protects_it`

688 **Shortage order needs and min stock.** 1
`test_shortage_order_needs_and_min_stock`

689 **Rights.** 1
Someone who may only view the Stock screen can't book or see the history.
`test_rights`

690 **Service never commits and never goes negative.** 1
`test_service_never_commits_and_never_goes_negative`

`test_wash_stats.py` 2

Atelier's wash statistics on the KPI page: pieces washed per Belgian day, linked to a festival by date (a day after the festival stays in its season, in the "no festival" group), undone washes left out, and the KPI right.

691 **Wash stats per day and festival.** 1
`test_wash_stats_per_day_and_festival`

692 **Wash stats needs the KPI right.** 1
`test_wash_stats_needs_the_kpi_right`

Walkies (module-6) 17 tests

Still a shell: access rights and test scans.

`test_help.py` 3

Walkies's help manual endpoint (GET /help/pdf): module access is needed, and a topic of a screen the user can't view is refused.

693 **Full manual and one topic.** 1
`test_full_manual_and_one_topic`

694 **Help needs module access.** 1
`test_help_needs_module_access`

695 **Topic of a hidden screen is not found.** 1
`test_topic_of_a_hidden_screen_is_not_found`

`test_roles.py` 14

These tests check Walkies's custom-roles-with-per-screen-permissions system — the same system Walkies uses (see tests/modules/module_6/test_roles.py), applied to module-6.

696 **Sync screens creates every defined screen.** 1
`test_sync_screens_creates_every_defined_screen`

697 **Sync screens is safe to run more than once.** 1
`test_sync_screens_is_safe_to_run_more_than_once`

698 **User without module access cannot view roles.** 1
`test_user_without_module_access_cannot_view_roles`

699 **User with access but no role cannot view roles screen.** 1
`test_user_with_access_but_no_role_cannot_view_roles_screen`

700 **Role with view but not edit cannot rename a role.** 1
`test_role_with_view_but_not_edit_cannot_rename_a_role`

701 **Super admin bypasses walkies permissions entirely.** 1
`test_super_admin_bypasses_walkies_permissions_entirely`

702 **Creating a role with a duplicate name is rejected.** 1
`test_creating_a_role_with_a_duplicate_name_is_rejected`

703 **Deleting a role still assigned to a user is rejected.** 1
`test_deleting_a_role_still_assigned_to_a_user_is_rejected`

704 **Set role permissions replaces the whole matrix.** 1
`test_set_role_permissions_replaces_the_whole_matrix`

705 **Permission on roles screen does not grant access to users screen.** 1
`test_permission_on_roles_screen_does_not_grant_access_to_users_screen`

706 **Permission on users screen does not grant access to roles screen.** 1
`test_permission_on_users_screen_does_not_grant_access_to_roles_screen`

707 **Cannot assign a role to a user without module access.** 1
`test_cannot_assign_a_role_to_a_user_without_module_access`

708 **Create or grant user creates a brand new account scoped to walkies only.** 1
`test_create_or_grant_user_creates_a_brand_new_account_scoped_to_walkies_only`

709 **Me permissions reflects only what the role allows.** 1
`test_me_permissions_reflects_only_what_the_role_allows`

Altsien Select (module-8) 90 tests

The team wizard, ploegfiche, copy from last season, mails.

`test_dashboard.py` 3

Tests for Altsien Select's KPI screen figures.

710 **Dashboard counts only the users teams.** 1
`test_dashboard_counts_only_the_users_teams`

711 **Dashboard marks a team complete when every step is done.** 1
`test_dashboard_marks_a_team_complete_when_every_step_is_done`

712 **Dashboard without a season is empty.** 1
`test_dashboard_without_a_season_is_empty`

`test_groundplans.py` 4

Tests for Altsien Select's read-only ground plan endpoints, which feed the map in Ploeg Wizard step 2 (afleverlocaties) — and for the coordinates the step's delivery-location options carry so they can be pinned on it.

713 **Wizard user can list and fetch ground plans.** 1
`test_wizard_user_can_list_and_fetch_ground_plans`

714 **Unknown ground plan image is 404 (not found).** 1
`test_unknown_ground_plan_image_is_404`

715 **User without wizard or ploegfiche view cannot read ground plans.** 1
`test_user_without_wizard_or_ploegfiche_view_cannot_read_ground_plans`

716 **Afleverlocatie options carry coordinates.** 1
`test_afleverlocatie_options_carry_coordinates`

`test_mails.py` 7

Altsien Select's mails: the "Instellingen > E-mails" endpoints (texts, placeholder check, test mail, CC list, rights) and the Ploegfiche PDF mail a Kernlid can ask for once every wizard step is done — to their own address, with the CC list in copy and the PDF attached. send_email is replaced by a collector, so nothing leaves the machine.

717 **Templates and placeholder check.** 1
`test_templates_and_placeholder_check`

718 **CC list is validated and normalized.** 1
`test_cc_list_is_validated_and_normalized`

719 **Test mail goes only to yourself.** 1
`test_test_mail_goes_only_to_yourself`

720 **Mail screen rights.** 1
A Kernlid without the screen sees nothing; view-only can't save.
`test_mail_screen_rights`

721 **Ploegfiche mail needs every step.** 1
`test_ploegfiche_mail_needs_every_step`

722 **Ploegfiche mail with PDF and CC.** 1
`test_ploegfiche_mail_with_pdf_and_cc`

723 **Ploegfiche mail scope and switched off.** 1
`test_ploegfiche_mail_scope_and_switched_off`

`test_ploegfiche.py` 5

Tests for Altsien Select's Ploegfiche (the one-screen overview of a team's choices) and its PDF.

724 **Ploegfiche lists the team karren.** 1
`test_ploegfiche_lists_the_team_karren`

725 **Ploegfiche shows every choice.** 1
`test_ploegfiche_shows_every_choice`

726 **Ploegfiche respects team scope.** 1
`test_ploegfiche_respects_team_scope`

727 **PDF is generated for an empty team.** 1
`test_pdf_is_generated_for_an_empty_team`

728 **PDF handles filled in data and non latin text.** 1
`test_pdf_handles_filled_in_data_and_non_latin_text`

`test_previous_season.py` 18

Tests for the Ploeg Wizard's "copy from last season": finding the season before, matching festivals by (fuzzy) name, flagging each line, copying delivery locations / team leads / special requests, and the usual scope, season-lock and validation rules.

729 **Previous season is the one before by name.** 1
`test_previous_season_is_the_one_before_by_name`

730 **Festival names match without case years and punctuation.** 1
`test_festival_names_match_without_case_years_and_punctuation`

731 **Steps say whether they offer a copy.** 1
`test_steps_say_whether_they_offer_a_copy`

732 **Step without copy returns 404 (not found).** 1
`test_step_without_copy_returns_404`

733 **No previous season gives no lines.** 1
`test_no_previous_season_gives_no_lines`

734 **Festival lines are matched and flagged.** 1
TW Classic is already chosen this season.
`test_festival_lines_are_matched_and_flagged`

735 **Copying festivals adds them and reopens the location step.** 1
`test_copying_festivals_adds_them_and_reopens_the_location_step`

736 **Inactive festival this season is not offered.** 1
`test_inactive_festival_this_season_is_not_offered`

737 **Location lines are matched and flagged.** 1
Only Rock Werchter is selected this season; TW Classic isn't.
`test_location_lines_are_matched_and_flagged`

738 **Inactive location cannot be copied.** 1
`test_inactive_location_cannot_be_copied`

739 **Copying locations writes plan a kar.** 1
`test_copying_locations_writes_plan_a_kar`

740 **Copying team leads adds them and skips duplicates.** 1
`test_copying_team_leads_adds_them_and_skips_duplicates`

741 **Same team lead twice last season is copied once.** 1
`test_same_team_lead_twice_last_season_is_copied_once`

742 **Copying requests creates new ones in the first status.** 1
`test_copying_requests_creates_new_ones_in_the_first_status`

743 **Same request text twice last season is copied once.** 1
`test_same_request_text_twice_last_season_is_copied_once`

744 **Unknown line key is refused.** 1
`test_unknown_line_key_is_refused`

745 **Cannot copy for another team.** 1
`test_cannot_copy_for_another_team`

746 **Cannot copy into a closed season.** 1
`test_cannot_copy_into_a_closed_season`

`test_products.py` 11

Tests for the Ploeg Wizard's Products step (step 4): the kar needs of the team's kars, stored in StockMaster's StockMaster_kar_requirement through module_4/requirement_service.py, but gated by Altsien Select's own rights, team scope and season lock.

747 **Save and read a kars needs.** 1
`test_save_and_read_a_kars_needs`

748 **Kars and catalog are limited to the team.** 1
`test_kars_and_catalog_are_limited_to_the_team`

749 **A kar of another team is not found.** 1
`test_a_kar_of_another_team_is_not_found`

750 **A team out of scope is not found.** 1
`test_a_team_out_of_scope_is_not_found`

751 **A closed season refuses changes.** 1
`test_a_closed_season_refuses_changes`

752 **View only users cannot save.** 1
`test_view_only_users_cannot_save`

753 **Invalid lines keep stockmasters error codes.** 1
`test_invalid_lines_keep_stockmasters_error_codes`

754 **Step needs at least one line and reopens when emptied.** 1
`test_step_needs_at_least_one_line_and_reopens_when_emptied`

755 **Previous season lines.** 1
`test_previous_season_lines`

756 **Excel template and import.** 1
`test_excel_template_and_import`

757 **Ploegfiche and PDF show the needs per kar.** 1
`test_ploegfiche_and_pdf_show_the_needs_per_kar`

`test_requests.py` 6

Tests for Altsien Select's special requests: the Kernlid adds them from the wizard (editable only while New), the organisation follows them up.

758 **New request starts in the first status.** 1
`test_new_request_starts_in_the_first_status`

759 **Blank request text is refused.** 1
`test_blank_request_text_is_refused`

760 **Kernlid can edit and delete while new.** 1
`test_kernlid_can_edit_and_delete_while_new`

761 **Kernlid cannot see the follow up list.** 1
`test_kernlid_cannot_see_the_follow_up_list`

762 **Organisation follow up locks the request for the kernlid.** 1
`test_organisation_follow_up_locks_the_request_for_the_kernlid`

763 **Status in use cannot be deleted.** 1
`test_status_in_use_cannot_be_deleted`

`test_responsibles.py` 6

Tests for the Ploeg Wizard's "Ploegverantwoordelijken" step: adding, changing and removing a team's responsible people from the wizard (stored in MasterData's MasterData_team_responsible, shared with its own screen), the team scope, the season lock, and the step's completion rule.

764 **Step sits between afleverlocaties and products.** 1
`test_step_sits_between_afleverlocaties_and_products`

765 **Kernlid can add change and remove a responsible.** 1
`test_kernlid_can_add_change_and_remove_a_responsible`

766 **Required fields and email format are enforced.** 1
`test_required_fields_and_email_format_are_enforced`

767 **Kernlid cannot touch another teams responsibles.** 1
`test_kernlid_cannot_touch_another_teams_responsibles`

768 **Closed season is read only.** 1
`test_closed_season_is_read_only`

769 **Step needs at least one responsible.** 1
`test_step_needs_at_least_one_responsible`

`test_roles.py` 15

These tests check Altsien Select's custom-roles-with-per-screen- permissions system — the same system Intervention Requests uses (see tests/modules/module_3/test_roles.py), applied to module-8. The wizard, Ploegfiche, requests and dashboard have their

own test files alongside.

770	**Sync screens creates every defined screen.** `test_sync_screens_creates_every_defined_screen`	1
771	**Sync screens is safe to run more than once.** `test_sync_screens_is_safe_to_run_more_than_once`	1
772	**User without module access cannot view roles.** `test_user_without_module_access_cannot_view_roles`	1
773	**User with access but no role cannot view roles screen.** `test_user_with_access_but_no_role_cannot_view_roles_screen`	1
774	**Role with view but not edit cannot rename a role.** `test_role_with_view_but_not_edit_cannot_rename_a_role`	1
775	**Super admin bypasses altsien select permissions entirely.** `test_super_admin_bypasses_altsien_select_permissions_entirely`	1
776	**Creating a role with a duplicate name is rejected.** `test_creating_a_role_with_a_duplicate_name_is_rejected`	1
777	**Deleting a role still assigned to a user is rejected.** `test_deleting_a_role_still_assigned_to_a_user_is_rejected`	1
778	**Set role permissions replaces the whole matrix.** `test_set_role_permissions_replaces_the_whole_matrix`	1
779	**Permission on roles screen does not grant access to users screen.** `test_permission_on_roles_screen_does_not_grant_access_to_users_screen`	1
780	**Permission on users screen does not grant access to roles screen.** `test_permission_on_users_screen_does_not_grant_access_to_roles_screen`	1
781	**Cannot assign a role to a user without module access.** `test_cannot_assign_a_role_to_a_user_without_module_access`	1
782	**Create or grant user creates a brand new account scoped to altsien select only.** `test_create_or_grant_user_creates_a_brand_new_account_scoped_to_altsien_select_only`	1
783	**Me permissions reflects only what the role allows.** `test_me_permissions_reflects_only_what_the_role_allows`	1
784	**User without wizard permission cannot open the wizard team list.** `test_user_without_wizard_permission_cannot_open_the_wizard_team_list`	1

`test_wizard.py` 15

Tests for Altsien Select's Ploeg Wizard: which teams a user may open, saving festivals and delivery locations (shared with KarTracker's "Plan a kar"), completing steps, and the season lock.

785	**Kernlid only sees their own teams.** `test_kernlid_only_sees_their_own_teams`	1
786	**Kernlid cannot open another team.** `test_kernlid_cannot_open_another_team`	1
787	**Organisation sees every team.** `test_organisation_sees_every_team`	1
788	**Saving festivals marks them selected.** `test_saving_festivals_marks_them_selected`	1
789	**Location is written to plan a kar.** `test_location_is_written_to_plan_a_kar`	1
790	**Location for an unselected festival is rejected.** `test_location_for_an_unselected_festival_is_rejected`	1
791	**Deselecting a festival removes its plan a kar row.** `test_deselecting_a_festival_removes_its_plan_a_kar_row`	1
792	**Festivals step needs at least one festival.** `test_festivals_step_needs_at_least_one_festival`	1
793	**Location step needs a location for every festival.** `test_location_step_needs_a_location_for_every_festival`	1

794	**Changing festivals reopens the location step.** `test_changing_festivals_reopens_the_location_step`	1
795	**Step without check can be marked done and reopened.** `test_step_without_check_can_be_marked_done_and_reopened`	1
796	**Removed walkies step no longer exists.** Walkie-talkies are not managed in Altsien Select (removed 2026-11-07). `test_removed_walkies_step_no_longer_exists`	1
797	**Unknown step returns 404 (not found).** `test_unknown_step_returns_404`	1
798	**Closed season is read only for a kernlid.** `test_closed_season_is_read_only_for_a_kernlid`	1
799	**Organisation can still edit a closed season.** `test_organisation_can_still_edit_a_closed_season`	1

MasterData (module-9) 145 tests

Seasons, festivals, teams, aliases, team responsibles, imports.

`test_dashboard.py` 2

These tests check MasterData's landing dashboard ("Masterdata Overview"): aggregate KPI stats gated by plain module access (no specific screen permission), mirroring backend/tests/modules/module_1/test_dashboard.py.

800	**Dashboard requires module access.** `test_dashboard_requires_module_access`	1
801	**Dashboard counts seasons.** `test_dashboard_counts_seasons`	1

`test_data_upload_download.py` 10

These tests check MasterData's "Data Upload/Download" screen: a single "masterdata.dataupload" screen key gates the XLSX template download, bulk import, and export endpoints for every MasterData table plus the team links (mirrors KarTracker's own tests/modules/module_2/test_kar_import.py / test_kar_status_import.py).

802	**Downloading the season template requires dataupload view permission.** `test_downloading_the_season_template_requires_dataupload_view_permission`	1
803	**Super admin can download the season template.** `test_super_admin_can_download_the_season_template`	1
804	**View only dataupload permission cannot import seasons.** `test_view_only_dataupload_permission_cannot_import_seasons`	1
805	**Season import creates new rows and rejects duplicates.** `test_season_import_creates_new_rows_and_rejects_duplicates`	1
806	**Season export contains every season.** `test_season_export_contains_every_season`	1
807	**Dataupload permission is independent of season screen permission.** `test_dataupload_permission_is_independent_of_season_screen_permission`	1
808	**Team import resolves location and rejects duplicate names.** `test_team_import_resolves_location_and_rejects_duplicate_names`	1
809	**Team export does not include task or kernlid columns.** `test_team_export_does_not_include_task_or_kernlid_columns`	1
810	**My permissions reports dataupload as creatable when granted.** `test_my_permissions_reports_dataupload_as_creatable_when_granted`	1
811	**My permissions does not report dataupload as creatable for view only user.** `test_my_permissions_does_not_report_dataupload_as_creatable_for_view_only_user`	1

`test_data_upload_download_links.py` 6

These tests check the parts of MasterData's "Data Upload/Download" added for keeping the uploads in line with the tables: updating a record through its "id" column (so an export can be edited and uploaded again), the new Season columns, and the

four team-link tiles (tasks, Kernleden and festivals per team, Ploegverantwoordelijken). Helpers come from test_data_upload_download.py, like the scanner tests do in module_1.

812	**Season periode open round trips and updates.** `test_season_periode_open_round_trips_and_updates`	1
813	**Team task links are added once and exported.** `test_team_task_links_are_added_once_and_exported`	1
814	**Kernlid links need an altsien kernlid user.** `test_kernlid_links_need_an_altsien_kernlid_user`	1
815	**Festival links follow the wizard rules.** `test_festival_links_follow_the_wizard_rules`	1
816	**Team responsibles are validated and updated by ID.** `test_team_responsibles_are_validated_and_updated_by_id`	1
817	**Link endpoints need a login.** `test_link_endpoints_need_a_login`	1

`test_delivery_methods.py` 11

These tests check the "Delivery Method" screen (nested under "Teams"): gated by normal module access + the "masterdata.delivery-method" permission, independently of "masterdata.roles"/"masterdata.users", with a super-admin bypass — mirrors test_season.py's shape.

818	**User without module access cannot list delivery methods.** `test_user_without_module_access_cannot_list_delivery_methods`	1
819	**User with access but no delivery method permission cannot list delivery methods.** `test_user_with_access_but_no_delivery_method_permission_cannot_list_delivery_methods`	1
820	**User with delivery method view permission can list delivery methods.** `test_user_with_delivery_method_view_permission_can_list_delivery_methods`	1
821	**Super admin can manage delivery methods without an explicit role.** `test_super_admin_can_manage_delivery_methods_without_an_explicit_role`	1
822	**Delivery method permission does not grant access to roles screen.** `test_delivery_method_permission_does_not_grant_access_to_roles_screen`	1
823	**View only delivery method permission cannot create a delivery method.** `test_view_only_delivery_method_permission_cannot_create_a_delivery_method`	1
824	**Creating a delivery method with a duplicate name is rejected.** `test_creating_a_delivery_method_with_a_duplicate_name_is_rejected`	1
825	**Super admin can rename a delivery method.** `test_super_admin_can_rename_a_delivery_method`	1
826	**Renaming a missing delivery method returns 404 (not found).** `test_renaming_a_missing_delivery_method_returns_404`	1
827	**Super admin can delete a delivery method.** `test_super_admin_can_delete_a_delivery_method`	1
828	**Deleting a missing delivery method returns 404 (not found).** `test_deleting_a_missing_delivery_method_returns_404`	1

`test_festivals.py` 15

These tests check the "Festivals" MasterData screen: gated by normal module access + the "masterdata.festival" permission, independently of "masterdata.season"/others, with a super-admin bypass, full CRUD, and season_id FK validation — mirrors test_scanners.py's shape.

829	**User without module access cannot list festivals.** `test_user_without_module_access_cannot_list_festivals`	1
830	**User with access but no festival permission cannot list festivals.** `test_user_with_access_but_no_festival_permission_cannot_list_festivals`	1
831	**Festival permission does not grant access to season screen.** `test_festival_permission_does_not_grant_access_to_season_screen`	1
832	**Super admin can create a festival without an explicit role.** `test_super_admin_can_create_a_festival_without_an_explicit_role`	1

833	**View only festival permission cannot create a festival.** `test_view_only_festival_permission_cannot_create_a_festival`	1
834	**View only festival permission cannot update or delete.** `test_view_only_festival_permission_cannot_update_or_delete`	1
835	**Creating a festival with a missing season ID returns 422 (invalid input).** `test_creating_a_festival_with_a_missing_season_id_returns_422`	1
836	**Creating a festival with an unknown season ID returns 404 (not found).** `test_creating_a_festival_with_an_unknown_season_id_returns_404`	1
837	**List festivals ordered by name.** `test_list_festivals_ordered_by_name`	1
838	**Super admin can update a festival.** `test_super_admin_can_update_a_festival`	1
839	**Festival is active by default and active can be set on create and update.** `test_festival_is_active_by_default_and_active_can_be_set_on_create_and_update`	1
840	**Updating a missing festival returns 404 (not found).** `test_updating_a_missing_festival_returns_404`	1
841	**Updating a festival with an unknown season ID returns 404 (not found).** `test_updating_a_festival_with_an_unknown_season_id_returns_404`	1
842	**Super admin can delete a festival.** `test_super_admin_can_delete_a_festival`	1
843	**Deleting a missing festival returns 404 (not found).** `test_deleting_a_missing_festival_returns_404`	1

`test_roles.py` 14

These tests check MasterData's custom-roles-with-per-screen-permissions system — the same system TagScan uses (see tests/modules/module_1/test_roles.py), applied to module-9. Season's own CRUD tests live in test_season.py alongside this file.

844	**Sync screens creates every defined screen.** `test_sync_screens_creates_every_defined_screen`	1
845	**Sync screens is safe to run more than once.** `test_sync_screens_is_safe_to_run_more_than_once`	1
846	**User without module access cannot view roles.** `test_user_without_module_access_cannot_view_roles`	1
847	**User with access but no role cannot view roles screen.** `test_user_with_access_but_no_role_cannot_view_roles_screen`	1
848	**Role with view but not edit cannot rename a role.** `test_role_with_view_but_not_edit_cannot_rename_a_role`	1
849	**Super admin bypasses MasterData permissions entirely.** `test_super_admin_bypasses_masterdata_permissions_entirely`	1
850	**Creating a role with a duplicate name is rejected.** `test_creating_a_role_with_a_duplicate_name_is_rejected`	1
851	**Deleting a role still assigned to a user is rejected.** `test_deleting_a_role_still_assigned_to_a_user_is_rejected`	1
852	**Set role permissions replaces the whole matrix.** `test_set_role_permissions_replaces_the_whole_matrix`	1
853	**Permission on roles screen does not grant access to users screen.** `test_permission_on_roles_screen_does_not_grant_access_to_users_screen`	1
854	**Permission on users screen does not grant access to roles screen.** `test_permission_on_users_screen_does_not_grant_access_to_roles_screen`	1
855	**Cannot assign a role to a user without module access.** `test_cannot_assign_a_role_to_a_user_without_module_access`	1
856	**Create or grant user creates a brand new account scoped to MasterData only.** `test_create_or_grant_user_creates_a_brand_new_account_scoped_to_masterdata_only`	1
857	**Me permissions reflects only what the role allows.** `test_me_permissions_reflects_only_what_the_role_allows`	1

test_season.py 14

These tests check the "Season" screen now that it lives inside MasterData: it's gated by normal module access + the "masterdata.season" permission (independently of "masterdata.roles"/"masterdata.users"), with a super-admin bypass, instead of the old flat "super admin only" check it used to have back when it lived under /api/admin/master-data.

858	**User without module access cannot list seasons.** `test_user_without_module_access_cannot_list_seasons`	1
859	**User with access but no season permission cannot list seasons.** `test_user_with_access_but_no_season_permission_cannot_list_seasons`	1
860	**User with season view permission can list seasons.** `test_user_with_season_view_permission_can_list_seasons`	1
861	**Super admin can manage seasons without an explicit role.** `test_super_admin_can_manage_seasons_without_an_explicit_role`	1
862	**Season permission does not grant access to roles screen.** `test_season_permission_does_not_grant_access_to_roles_screen`	1
863	**View only season permission cannot create a season.** `test_view_only_season_permission_cannot_create_a_season`	1
864	**Creating a season with a duplicate name is rejected.** `test_creating_a_season_with_a_duplicate_name_is_rejected`	1
865	**Creating a season with periode open round trips it.** `test_creating_a_season_with_periode_open_round_trips_it`	1
866	**Creating a season without periode open defaults to false.** `test_creating_a_season_without_periode_open_defaults_to_false`	1
867	**Super admin can rename a season.** `test_super_admin_can_rename_a_season`	1
868	**Super admin can toggle a seasons periode open.** `test_super_admin_can_toggle_a_seasons_periode_open`	1
869	**Renaming a missing season returns 404 (not found).** `test_renaming_a_missing_season_returns_404`	1
870	**Super admin can delete a season.** `test_super_admin_can_delete_a_season`	1
871	**Deleting a missing season returns 404 (not found).** `test_deleting_a_missing_season_returns_404`	1

test_team_aliases.py 5

Team aliases: set on the team endpoints and the Teams XLSX, refused when they clash with another team, and used to match a wristband scan's team name.

872	**Aliases are stored cleaned and kept when left out.** `test_aliases_are_stored_cleaned_and_kept_when_left_out`	1
873	**Alias clash with another team is refused.** `test_alias_clash_with_another_team_is_refused`	1
874	**Match team uses name then alias.** `test_match_team_uses_name_then_alias`	1
875	**Deleting a team frees its aliases.** `test_deleting_a_team_frees_its_aliases`	1
876	**XLSX import and export.** `test_xlsx_import_and_export`	1

test_team_festivals.py 11

Tests for MasterData's "Festivals per ploeg" grid (masterdata.team-festivals): the grid of one year, saving ticked/unticked cells through the shared team_festival_service (removing a link deletes the Plan a kar location and re-opens Altsien Select's "afleverlocaties" step), the closed-year lock, the rights, and the "copy from previous year" proposal.

877	**Grid lists festivals teams and links.** `test_grid_lists_festivals_teams_and_links`	1
878	**Options default to the last open year.** `test_options_default_to_the_last_open_year`	1

879	**Ticking adds links and reopens the location step.** `test_ticking_adds_links_and_reopens_the_location_step`	1
880	**Unticking deletes the delivery location.** `test_unticking_deletes_the_delivery_location`	1
881	**Inactive festival cannot be ticked.** `test_inactive_festival_cannot_be_ticked`	1
882	**Festival of another year is refused.** `test_festival_of_another_year_is_refused`	1
883	**Closed year is read only.** `test_closed_year_is_read_only`	1
884	**View right cannot save.** `test_view_right_cannot_save`	1
885	**No right cannot view.** `test_no_right_cannot_view`	1
886	**Previous year is mapped by festival name.** `test_previous_year_is_mapped_by_festival_name`	1
887	**No previous year gives an empty proposal.** `test_no_previous_year_gives_an_empty_proposal`	1

`test_team_locations.py` 11

These tests check the "Team Location" screen (nested under "Teams"): gated by normal module access + the "masterdata.team-location" permission, independently of "masterdata.roles"/"masterdata.users", with a super-admin bypass — mirrors test_season.py's shape.

888	**User without module access cannot list team locations.** `test_user_without_module_access_cannot_list_team_locations`	1
889	**User with access but no team location permission cannot list team locations.** `test_user_with_access_but_no_team_location_permission_cannot_list_team_locations`	1
890	**User with team location view permission can list team locations.** `test_user_with_team_location_view_permission_can_list_team_locations`	1
891	**Super admin can manage team locations without an explicit role.** `test_super_admin_can_manage_team_locations_without_an_explicit_role`	1
892	**Team location permission does not grant access to roles screen.** `test_team_location_permission_does_not_grant_access_to_roles_screen`	1
893	**View only team location permission cannot create a team location.** `test_view_only_team_location_permission_cannot_create_a_team_location`	1
894	**Creating a team location with a duplicate name is rejected.** `test_creating_a_team_location_with_a_duplicate_name_is_rejected`	1
895	**Super admin can rename a team location.** `test_super_admin_can_rename_a_team_location`	1
896	**Renaming a missing team location returns 404 (not found).** `test_renaming_a_missing_team_location_returns_404`	1
897	**Super admin can delete a team location.** `test_super_admin_can_delete_a_team_location`	1
898	**Deleting a missing team location returns 404 (not found).** `test_deleting_a_missing_team_location_returns_404`	1

`test_team_responsibles.py` 10

These tests check the "Ploegverantwoordelijken" screen (nested under "Teams"): gated by normal module access + the "masterdata.team-responsibles" permission, with a super-admin bypass — mirrors test_team_locations.py's shape, plus the team/season links.

899	**User with access but no permission cannot list team responsibles.** `test_user_with_access_but_no_permission_cannot_list_team_responsibles`	1
900	**Viewer can list and filter team responsibles.** `test_viewer_can_list_and_filter_team_responsibles`	1

901	**View only permission cannot create a team responsible.** `test_view_only_permission_cannot_create_a_team_responsible`	1
902	**Super admin can create update and delete a team responsible.** `test_super_admin_can_create_update_and_delete_a_team_responsible`	1
903	**Required fields and email format are enforced.** `test_required_fields_and_email_format_are_enforced`	1
904	**Unknown team or season is a 404 (not found).** `test_unknown_team_or_season_is_a_404`	1
905	**Deleting a team removes its responsibles.** `test_deleting_a_team_removes_its_responsibles`	1
906	**Moving the last responsible away reopens the wizard step.** `test_moving_the_last_responsible_away_reopens_the_wizard_step`	1
907	**Moving one of two responsibles keeps the wizard step.** `test_moving_one_of_two_responsibles_keeps_the_wizard_step`	1
908	**Deleting the last responsible reopens the wizard step.** `test_deleting_the_last_responsible_reopens_the_wizard_step`	1

`test_team_tasks.py` 11

These tests check the "Team Tasks" screen (nested under "Teams"): gated by normal module access + the "masterdata.team-tasks" permission, independently of "masterdata.roles"/"masterdata.users", with a super-admin bypass — mirrors test_season.py's shape.

909	**User without module access cannot list team tasks.** `test_user_without_module_access_cannot_list_team_tasks`	1
910	**User with access but no team task permission cannot list team tasks.** `test_user_with_access_but_no_team_task_permission_cannot_list_team_tasks`	1
911	**User with team task view permission can list team tasks.** `test_user_with_team_task_view_permission_can_list_team_tasks`	1
912	**Super admin can manage team tasks without an explicit role.** `test_super_admin_can_manage_team_tasks_without_an_explicit_role`	1
913	**Team task permission does not grant access to roles screen.** `test_team_task_permission_does_not_grant_access_to_roles_screen`	1
914	**View only team task permission cannot create a team task.** `test_view_only_team_task_permission_cannot_create_a_team_task`	1
915	**Creating a team task with a duplicate name is rejected.** `test_creating_a_team_task_with_a_duplicate_name_is_rejected`	1
916	**Super admin can rename a team task.** `test_super_admin_can_rename_a_team_task`	1
917	**Renaming a missing team task returns 404 (not found).** `test_renaming_a_missing_team_task_returns_404`	1
918	**Super admin can delete a team task.** `test_super_admin_can_delete_a_team_task`	1
919	**Deleting a missing team task returns 404 (not found).** `test_deleting_a_missing_team_task_returns_404`	1

`test_teams.py` 25

These tests check the "Teams" screen: gated by normal module access + the "masterdata.teams" permission, independently of "masterdata.roles"/"masterdata.users", with a super-admin bypass — mirrors test_season.py's/test_team_locations.py's shape, plus extra coverage for Teams' FK/many-to-many fields (location, delivery method, tasks, kernleden).

920	**User without module access cannot list teams.** `test_user_without_module_access_cannot_list_teams`	1
921	**User with access but no teams permission cannot list teams.** `test_user_with_access_but_no_teams_permission_cannot_list_teams`	1
922	**User with teams view permission can list teams.** `test_user_with_teams_view_permission_can_list_teams`	1
923	**Super admin can manage teams without an explicit role.** `test_super_admin_can_manage_teams_without_an_explicit_role`	1

924 **Teams permission does not grant access to roles screen.** 1
`test_teams_permission_does_not_grant_access_to_roles_screen`

925 **View only teams permission cannot create a team.** 1
`test_view_only_teams_permission_cannot_create_a_team`

926 **Creating a team with a duplicate name is rejected.** 1
`test_creating_a_team_with_a_duplicate_name_is_rejected`

927 **Super admin can rename a team.** 1
`test_super_admin_can_rename_a_team`

928 **Team is active by default and active can be set on create and update.** 1
`test_team_is_active_by_default_and_active_can_be_set_on_create_and_update`

929 **Renaming a missing team returns 404 (not found).** 1
`test_renaming_a_missing_team_returns_404`

930 **Super admin can delete a team.** 1
`test_super_admin_can_delete_a_team`

931 **Deleting a missing team returns 404 (not found).** 1
`test_deleting_a_missing_team_returns_404`

932 **Creating a team with valid location delivery method tasks and kernleden.** 1
`test_creating_a_team_with_valid_location_delivery_method_tasks_and_kernleden`

933 **Creating a team with an unknown location ID returns 404 (not found).** 1
`test_creating_a_team_with_an_unknown_location_id_returns_404`

934 **Creating a team with an unknown delivery method ID returns 404 (not found).** 1
`test_creating_a_team_with_an_unknown_delivery_method_id_returns_404`

935 **Creating a team with an unknown task ID returns 404 (not found).** 1
`test_creating_a_team_with_an_unknown_task_id_returns_404`

936 **Creating a team with an unknown kernlid ID returns 404 (not found).** 1
`test_creating_a_team_with_an_unknown_kernlid_id_returns_404`

937 **Updating a team with an unknown task ID returns 404 (not found).** 1
`test_updating_a_team_with_an_unknown_task_id_returns_404`

938 **Updating a team replaces its task links rather than appending.** 1
`test_updating_a_team_replaces_its_task_links_rather_than_appending`

939 **Creating a team with a user who is not flagged altsien kernlid returns 404 (not found).** 1
`test_creating_a_team_with_a_user_who_is_not_flagged_altsien_kernlid_returns_404`

940 **Updating a team replaces its kernlid links rather than appending.** 1
`test_updating_a_team_replaces_its_kernlid_links_rather_than_appending`

941 **Deleting a team cascades its join table rows.** 1
`test_deleting_a_team_cascades_its_join_table_rows`

942 **A team saves and returns its special tours.** 1
`test_a_team_saves_and_returns_its_special_tours`

943 **An unknown special tour ID returns 404 (not found).** 1
`test_an_unknown_special_tour_id_returns_404`

944 **Special tour options need only the teams view right.** 1
`test_special_tour_options_need_only_the_teams_view_right`

Mobile App (module-10) 3 tests

The phone-app install page.

`test_install_info.py` 3

These tests check the phone-app install page's data endpoint (module-10): it needs a login, and it returns the phone section's (/m) address and a QR code of it, built from the environment's APP_PUBLIC_URL — or nothing when that isn't configured.

945 **Install info requires login.** 1
`test_install_info_requires_login`

946 **Install info is empty without a public URL.** 1
`test_install_info_is_empty_without_a_public_url`

947 **Install info points at the phone section.** 1
A trailing slash on the setting must not give a double slash.
`test_install_info_points_at_the_phone_section`